# Package 'jjstat'

October 21, 2026

**Type** Package

**Title** Statistical Analysis Tools for Writing Papers

**Version** 0.1.9

**Author** Park Joonghee

**Maintainer** The package maintainer <shoutjoy@hanmail.net>

**Description** More about what it does (maybe more than one line)
Use four spaces when indenting paragraphs within the Description.

**License** GPL-3

**Encoding** UTF-8

**LazyData** true

**Imports** dplyr ,
knitr ,
ggplot2 (>= 3.4.4),
magrittr (>= 2.0.3),
Hmisc,
ggrepel

**Suggests** broom (>= 1.0.5),
httpuv (>= 1.6.13),
rvest (>= 1.0.3),
tibble (>= 3.2.1),
kableExtra (>= 1.3.4),
testthat (>= 3.0.0),
psych,
gridExtra

**Depends** tidyverse,
lavaan,
broom,
plspm,
R (>= 2.10),

**Config/roxygen2/version** 8.1.1

## R topics documented:

| | |
|---|---|
| accent_table | *aaccent_table: table() analytics as data frame, mosaicplot, usign add_ratio* |

## Description

aaccent_table: table() analytics as data frame, mosaicplot, usign add_ratio

## Usage

```
accent_table(
  data,
  cols = "a1",
  rows = "성조",
  title = "",
  trans = TRUE,
  cex = 1.3,
  color = TRUE,
  ylab = "onset",
  xlab = "accent",
  sub = NULL,
  plot = FALSE,
  type = "res",
  raw = FALSE
)
```

## Arguments

| | |
|---|---|
| data | table data |
| cols | column names in data |
| rows | row names in data |
| title | table main |
| trans | transpose |
| cex | graph text size |
| color | graph color 1:2, mosaicplot |
| ylab | ylab |
| xlab | xlab |
| sub | subtitle |
| plot | output option |
| type | res, res_df, ratio, g, all |
| raw | raw data, or table data |

## Value

matrix type data.frame

### Examples

```
\#\# Not run:
mtcars %>% select(am, vs) %>% table() %>% accent_table("am","vs")

mtcars %>% select(am, vs) %>% table() %>% accent_table("am","vs", plot=TRUE)


mtcars %>% select(am, vs) %>% table() %>% accent_table("am","vs", plot=TRUE, title = "Mtcars")


## End(Not run)
```

---

`add_bind_stat` *Functions that item parceling generate multiple pieces of data,*

---

### Description

Functions that item parceling generate multiple pieces of data,

### Usage

```
add_bind_stat(data, ..., fun = mean, type = "res")
```

### Arguments

| | |
|---|---|
| `data` | data.fram |
| `...` | colname, cols |
| `fun` | fun mean sd |
| `type` | data, res –data.fram , res, cols_check |

### Value

data.frame

### Examples

```
\#\# Not run:

#Common item parceling methods (group by average )
mtcars %>% mutate(
Mean_col1 = (mpg + cyl)/2 ,
Mean_col2 = (disp + hp)/2 ,
Mean_col3 = (drat + wt )/2 ,
Mean_col4 = (qsec + vs + am)/3 )

#Methods for item parceling using functions
add_bind_stat(mtcars,
c("Mean_col1","mpg", "cyl"),
c("Mean_col2", "disp", "hp"),
```

```
c("Mean_col3", "drat", "wt"),
c("Mean_col4", "qsec", "vs", "am") )



add_bind_stat(mtcars,
   c("Mean_col1", 1:2),
   c("Mean_col2", 3:4),
   c("Mean_col3", 5:6),
   c("Mean_col4", 7:9))
#'  ##col_check
#'
add_bind_stat(mtcars,
              c("Mean_col1", "mpg:cyl"),
              c("Mean_col2", "disp:hp"),
              c("Mean_col3", "drat:wt"),
              c("Mean_col4", "qsec:am"))

## End(Not run)
```

---

`add_col_ratio` 그룹합에비율을만들어주는함수

---

## Description

그룹합에비율을만들어주는함수

## Usage

```
add_col_ratio(data, col, n_col)
```

## Arguments

| | |
|---|---|
| `data` | data |
| `col` | 기준이되는열 |
| `n_col` | 빈도를나타내는열 |

## Value

df

## Examples

```
\#\# Not run:
#'
# 사용 예제
library(dplyr)

# 주어진 데이터프레임
data <- data.frame(
  Q17 = c("", "고1 남학생", "고1 여학생", "고2 남학생", "고2 여학생",
          "고3 남학생", "고3 여학생", "중1 남학생", "중1 여학생",
          "중2 남학생", "중2 여학생", "중3 남학생", "중3 여학생",
```

```
          "초1 남학생", "초1 여학생", "초2 남학생", "초2 여학생",
          "초3 남학생", "초3 여학생", "초4 남학생", "초4 여학생",
          "초5 남학생", "초5 여학생", "초6 남학생", "초6 여학생"),
  Q16 = c("", rep("", 24)),
  n = c(1130, 413, 184, 329, 163, 216, 72, 354, 323, 370, 308,
        195, 169, 7, 4, 25, 23, 60, 53, 174, 162, 237, 216, 307, 293)
)

# 함수 적용: 열 이름으로 지정
result1 <- add_col_ratio(data, col = "Q17", n_col = "n")

# 함수 적용: 숫자 인덱스로 지정
result2 <- add_col_ratio(data, col = 1, n_col = 3)

# 결과 확인
print(result1)
print(result2)



## End(Not run)
```

---

`add_cor` *correlation analysis*

---

## Description

correlation analysis

## Usage

```
add_cor(selectdata, type = "res_all", digits = 3, plot = TRUE, md = FALSE)
```

## Arguments

| | |
|---|---|
| `selectdata` | seledted daa |
| `type` | res_all, res, p, r, res_p, res_md, res_lower, g(chart.Correlation) |
| `digits` | default 3 |
| `plot` | graph performanceAnalytis::chart.Correlation |
| `md` | markdown table output |

## Value

table and picture

## Examples

```
\#\# Not run:

mtcars%>%select(mpg, disp, wt, hp, qsec)%>%add_cor()

## End(Not run)
```

`add_id` *Add row id*

## Description

Add row id

## Usage

```
add_id(data, sel = "names", each = TRUE)
```

## Arguments

| | |
|---|---|
| `data` | data.frame |
| `sel` | select column |
| `each` | each =TRUE then 111222....FALSE 123123 |

## Value

add column data

## Examples

```
\#\# Not run:

mtcars %>% mutate(MPG= which(colnames(mtcars) =="mpg"))%>%add_id()
mtcars %>% mutate(MPG= which(colnames(mtcars) =="mpg"))%>%add_id("cyl")
mtcars[1:3, 1:3] %>% add_id()
mtcars[1:3, 1:3] %>% add_id("cyl")
mtcars[1:4, 1:4] %>% to_long() %>% add_id()
mtcars[1:4, 1:4] %>% to_long() %>% add_id(each=FALSE)



## End(Not run)
```

`add_normality` *Add skewness and kurtosis normality checks to descriptive statistics results*

## Description

Add skewness and kurtosis normality checks to descriptive statistics results

## Usage

```
add_normality(data, skew = "Skew", kurt = "Kurt")
```

### Arguments

| | |
|---|---|
| data | mysummary data |
| skew | select skew |
| kurt | select kurt |

### Value

normalitydata

### Examples

```
\#\# Not run:

mysummary(mtcars)
mysummary(mtcars)%>%add_normality()



## End(Not run)
```

---

## add_normality_kline *Add Normality Kline*

---

### Description

This function evaluates normality for each variable in the dataset based on skewness and kurtosis.

### Usage

```
add_normality_kline(df, skew = 3, kurt = 8)
```

### Arguments

| | |
|---|---|
| df | A data frame containing statistical summary including Skew and Kurt columns. |
| skew | Threshold for skewness. Default is 3.0. |
| kurt | Threshold for kurtosis. Default is 8.0. |

### Value

A data frame with added columns ‘skew_sig‘ and ‘kurt_sig‘ indicating the normality evaluation.

### Examples

```
\#\# Not run:
data <- data.frame(
  var = c("SUX", "FLOW", "PU", "PEOU", "ATT", "USE"),
  N = c(895, 895, 895, 895, 895, 895),
  MEAN = runif(6, 2, 3),
  SD = runif(6, 0.9, 1.1),
  MIN = rep(1, 6),
  MAX = rep(5, 6),
  Skew = c(0.74, 0.23, 0.81, 0.75, 0.69, 0.53),
  Kurt = c(0.16, -0.31, 0.37, 0.06, 0.02, -0.42)
)
add_normality_kline(data)

## End(Not run)
```

---

## add_partial_sum *add_partial_sum*

---

### Description

add_partial_sum

### Usage

```
add_partial_sum(df, col, freq_col)
```

### Arguments

| | |
|---|---|
| df | df |
| col | select col |
| freq_col | select count frequency col |

### Value

df

### Examples

```
\#\# Not run:
#'
# 함수 사용 예시
df <- tibble::tribble(
  ~학교급, ~term, ~Freq, ~`prop(%)`,
  "-", "합계", 3110, 100,
  "고등학교", "고1", 341, 11.0,
  "고등학교", "고2", 322, 10.4,
  "고등학교", "고3", 192, 6.17,
  "중학교", "중1", 591, 19.0,
  "중학교", "중2", 527, 16.9,
  "중학교", "중3", 290, 9.32,
  "초등학교", "초1", 8, 0.257,
  "초등학교", "초2", 48, 1.54,
```

```
  "초등학교", "초3", 47, 1.51,
  "초등학교", "초4", 191, 6.14,
  "초등학교", "초5", 207, 6.66,
  "초등학교", "초6", 346, 11.1
)

# 함수 실행
result_df <- add_partial_sum(df, col = "학교급", freq_col = "Freq")
print(result_df)

## End(Not run)
```

---

`add_pca_scores` *add_pca_scores*

---

## Description

add_pca_scores

## Usage

```
add_pca_scores(data, ..., method = "nipals", adjust = NULL)
```

## Arguments

| | |
|---|---|
| `data` | data |
| `...` | select variable |
| `method` | niplas, prcomp |
| `adjust` | Options to change the sign of prcomp c(F, T ...) |

## Value

add data

## Examples

```
\#\# Not run:
data(offense)

nfl_path = matrix(
  c(0, 0, 0, 0, 0,
    0, 0, 0, 0, 0,
    0, 0, 0, 0, 0,
    0, 1, 1, 0, 0,
    1, 0, 0, 1, 0),
  nrow = 5, ncol = 5, byrow = TRUE )

rownames(nfl_path) <- c("Speical", "Rushing", "Passing", "Offense", "Scoring")
colnames(nfl_path) <- c("Speical", "Rushing", "Passing", "Offense", "Scoring")
nfl_path
Offense = offense %>% add_pca_scores(rush1 = c(1:3), pass1 = c(4:6))
```

```
Offense =  add_pca_scores(offense,
               rush1 = c("YardsRushAtt", "RushYards", "RushFirstDown"),
               pass1 = c("YardsPassComp", "PassYards", "PassFirstDown"))


nfl_blocks2 = list(7:8, 1:3, 4:6,  18:19, 9:11)

plspm_blocks_match(data=Offense, blocks=nfl_blocks2,
                   paths=nfl_path)

plspm_blocks_match(data=Offense, blocks=nfl_blocks2,
                   paths=nfl_path)%>%make_list_text()

nfl_blocks2 = list(
  Speical = c("FieldGoals", "OtherTDs"),
  Rushing = c("YardsRushAtt", "RushYards", "RushFirstDown"),
  Passing = c("YardsPassComp", "PassYards", "PassFirstDown"),
  Offense = c("rush1", "pass1"),
  Scoring = c("PointsGame", "OffensTD", "TDGame")
)
nfl_blocks2
nfl_mode2 =c("B","A","A","A","A")
nfl_pls2 = plspm(Data=Offense, nfl_path, nfl_blocks2, modes = nfl_mode2)
nfl_pls2%>%summary()

nfl_pls_boot2 = plspm_sem(Data=Offense, nfl_path, nfl_blocks2, modes = nfl_mode)
x11()
nfl_pls_boot2 %>%plspm_path_coefs_plot()
nfl_pls_boot2 %>%plspm_semPaths2()

#'
#'

# 함수 테스트
offense <- data.frame(
  YardsRushAtt = rnorm(10),
  RushYards = rnorm(10),
  RushFirstDown = rnorm(10),
  YardsPassComp = rnorm(10),
  PassYards = rnorm(10),
  PassFirstDown = rnorm(10)
)

# 테스트 케이스 1
offense1 <- add_pca_scores(offense,
                           rush1 = c("YardsRushAtt", "RushYards", "RushFirstDown"),
                           pass1 = c("YardsPassComp", "PassYards", "PassFirstDown"))

str(offense1)

# 테스트 케이스 2
offense2 <- add_pca_scores(offense, method = "prcomp",
                           rush1 = c("YardsRushAtt", "RushYards", "RushFirstDown"),
                           pass1 = c("YardsPassComp", "PassYards", "PassFirstDown"))

str(offense2)
```

```
# 테스트 케이스 3 (adjust 옵션 포함)
offense3 <- add_pca_scores(offense, method = "prcomp", adjust = c(F, T),
                           rush1 = c("YardsRushAtt", "RushYards", "RushFirstDown"),
                           pass1 = c("YardsPassComp", "PassYards", "PassFirstDown"))

str(offense3)
#'
#'


## End(Not run)
```

---

`add_prop` *Add Proportions to Frequency Data*

---

## Description

This function adds a column to a data frame representing the proportion of each frequency value, either as a percentage or a simple ratio.

## Usage

```
add_prop(df, freq = "Freq", prop = TRUE)
```

## Arguments

| | |
|---|---|
| `df` | A data frame containing frequency data. |
| `freq` | A column name or index representing the frequency column. Defaults to "Freq". If a numeric value is provided, it is converted to the corresponding column name. |
| `prop` | Logical, whether to return proportions as percentages (TRUE, default) or ratios (FALSE). |

## Value

A data frame with an additional column for proportions. If 'prop = TRUE', the column is named 'prop( If 'prop = FALSE', the column is named 'prop'.

## Examples

```
\#\# Not run:
# Example data
df <- data.frame(Category = c("A", "B", "C"), Freq = c(30, 50, 20))

# Adding proportions as percentages
add_prop(df, freq = "Freq", prop = TRUE)

# Adding proportions as simple ratios
add_prop(df, freq = "Freq", prop = FALSE)

## End(Not run)
```

`add_ratio` *Convert matrix data to a ratio by using the sum of columns to get a ratio*

## Description

Convert matrix data to a ratio by using the sum of columns to get a ratio

## Usage

```
add_ratio(data, type = "res2", digits = 3)
```

## Arguments

| | |
|---|---|
| `data` | matrix |
| `type` | res2: *100, res1 |
| `digits` | defalut 3 |

## Value

ratio tabl e

## Examples

```
\#\# Not run:


mat = matrix(c(36, 67, 11,
          0, 35, 44,
         41, 108, 1,
          56, 87, 54),
           nrow = 4, byrow = TRUE,
 dimnames = list(c("공명음", "마찰음",
                   "유기음_경음", "평파열음_평파찰음"),
                    c("H", "H(H)", "L")))

## step1
  mat %>% add_ratio()
##conbind
 mat %>% add_ratio_df()
##add margins
 mat %>% add_sum()


## End(Not run)
```

## add_ratio_df *Combine matrices and their ratios*

### Description

Combine matrices and their ratios

### Usage

```
add_ratio_df(res)
```

### Arguments

| | |
|---|---|
| res | matrix |

### Value

combine table /matrix

### Examples

```
\#\# Not run:

#' #'mat = matrix(c(36, 67, 11,
          0, 35, 44,
         41, 108, 1,
          56, 87, 54),
           nrow = 4, byrow = TRUE,
   dimnames = list(c("공명음", "마찰음",
     "유기음_경음", "평파열음_평파찰음"),  c("H", "H(H)", "L")))

## step1
  mat %>% add_ratio()
##conbind
 mat %>% add_ratio_df()
##add margins
 mat %>% add_sum()

## End(Not run)
```

## add_rep_id *Create an ordered variable for overlapping groups*

### Description

Create an ordered variable for overlapping groups

### Usage

```
add_rep_id(data, grp)
```

### Arguments

| | |
|---|---|
| data | data.frame |
| grp | select colname or number |

### Value

create data

### Examples

```
\#\# Not run:
#'
data <- data.frame(
  weight = c(4.17, 5.58, 5.18, 6.11, 4.50, 4.61, 5.17, 4.53, 5.33, 5.14,
             4.81, 4.17, 4.41, 3.59, 5.87, 3.83, 6.03, 4.89, 4.32, 4.69,
             6.31, 5.12, 5.54, 5.50, 5.37, 5.29, 4.92, 6.15, 5.80, 5.26),
  group = c(rep("ctrl", 10), rep("trt1", 10), rep("trt2", 10))
)
data

#generate id from Colname
add_rep_id(data, "group")

#generate id from Col number
add_rep_id(data, 2)

#data spread wide
data%>%add_rep_id("group")%>%as_trt("id") %>%wide_df("group", "weight")


data%>%
  add_rep_id("group")%>%
  as_trt("id") %>%
  wide_df("group", "weight")


## End(Not run)
```

---

## add_rows *Insert data into the required act in the dataframe*

---

### Description

Insert data into the required act in the dataframe

### Usage

```
add_rows(df, ..., row = NULL, char = FALSE)
```

### Arguments

| | |
|---|---|
| df | data.frame |
| ... | data, c(100, 200), c(200, 300), |
| row | row position |
| char | transpose character column |

### Value

dagta

### Examples

```
\#\# Not run:
#'
#'


df <- data.frame(A = 1:5, B = 6:10)
df
# 2번째 행부터 새 데이터(벡터 c(1,2,3,4)와 c(2,3,4,5))를 삽입

add_rows(df, c(100, 200), c(200, 300), row = 2)

df <- data.frame(
  측정변수 = c("진로목표(L01)", "준비여부(L02)", "직업결정(L03)", "부모추천(L04)", "학업성적(L05)"),
  Class.1 = c(0.2178, 0.3533, 0.1552, 0.2708, 0.1605),
  Class.2 = c(0.9888, 0.9769, 0.9910, 1.0000, 0.9804),
  Class.3 = c(0.9122, 0.8704, 0.9667, 0.0000, 0.6761)
)

# 1행에 새 데이터를 삽입
new_df <- add_rows(df, c("class name", "불명확 진로인식형", "적극적 진로준비형", "조건분 진로지향형")

print(new_df)


df <- data.frame(
  측정변수 = c("진로목표(L01)", "준비여부(L02)", "직업결정(L03)", "부모추천(L04)", "학업성적(L05)"),
  Class.1 = c(0.2178, 0.3533, 0.1552, 0.2708, 0.1605),
  Class.2 = c(0.9888, 0.9769, 0.9910, 1.0000, 0.9804),
  Class.3 = c(0.9122, 0.8704, 0.9667, 0.0000, 0.6761)
)

# 첫 번째 행에 새 데이터를 삽입, char = TRUE로 문자형 변환
new_df_1 <- add_rows(df, c("class name", "불명확 진로인식형", "적극적 진로준비형", "조건분 진로지향형

print(new_df_1)

# 마지막 행에 새 데이터를 삽입, char = FALSE로 기존 타입 유지
new_df_2 <- add_rows(df, c("class name", "불명확 진로인식형", "적극적 진로준비형", "조건분 진로지향형

print(new_df_2)
```

```
## End(Not run)
```

---

`add_rows_freq` *Functions that generate rows with a frequency*

---

## Description

Functions that generate rows with a frequency

## Usage

```
add_rows_freq(data, type = "res", sel = ncol(data), fix = NULL)
```

## Arguments

| | |
|---|---|
| `data` | data |
| `type` | res, all |
| `sel` | frequency column |
| `fix` | final select column |

## Value

data

## Examples

```
\#\# Not run:
dff = data.frame(
  trt = c("r","s","z"),
  con = c("f","m","l"),
  num = c(10,0,2),
  fre = c(4,0,5) )
dff
dff %>%add_rows_freq()
dff %>%add_rows_freq(sel=4)
dff %>%add_rows_freq(sel="fre")
dff %>%add_rows_freq(sel=3)
dff %>%add_rows_freq(sel="num")
dff %>%add_rows_freq(fix=1)
dff %>%add_rows_freq(type= "res", fix=1:2)
dff %>%add_rows_freq(type= "res", fix="con")
dff %>%add_rows_freq(type= "all")
dff %>%add_rows_freq(type= "real")
dff %>%add_rows_freq(type= "zero")
dff %>%add_rows_freq(type= "zero", fix="con")


# Making frequency data into real data
dff %>%to_long()%>% add_rows_freq()
 dff_real =  dff %>%to_long()%>% add_rows_freq()
 dff_real
```

```
freq_data =  mtcars %>% select(am, vs) %>% table()
freq_data%>% data.frame()
freq_data%>% data.frame() %>% add_rows_freq(type ="zero")
freq_data%>% data.frame() %>% add_rows_freq(type ="zero") %>%table()


## End(Not run)
```

---

`add_row_sum` *Add Row Sum to Data Frame*

---

### Description

This function calculates the sum of specified numeric columns and appends a summary row to the input data frame. The summary row includes totals for ‘freq_col‘ and ‘prop_col‘.

### Usage

```
add_row_sum(df, freq_col = 2, prop_col = 3)
```

### Arguments

| | |
|---|---|
| `df` | A data frame containing the data to be summed. |
| `freq_col` | Column name or index for the frequency column. Default is 2. |
| `prop_col` | Column name or index for the proportion column. Default is 3. |

### Value

A data frame with an added summary row containing the sums.

### Examples

```
\#\# Not run:
data <- data.frame(
  Group = c("A", "B", "C"),
  Freq = c(10, 20, 30),
  `prop(%)` = c(25.0, 50.0, 25.0)
)
add_row_sum(data, freq_col = "Freq", prop_col = "prop(%)")

## End(Not run)
```

## add_sig *mutate data generate*

### Description

mutate data generate

### Usage

```
add_sig(var)
```

### Arguments

| | |
|---|---|
| p.value | pvalue |

### Value

in data

### Examples

```
\#\# Not run:

aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"), grp_mean = T,
unite_F=TRUE) %>%
 mutate(STAR= add_sig(p_value))


## End(Not run)
```

## add_significance_symbols *Functions that output statistical significance for matrices and dataframes Simple and exact expressions*

### Description

Functions that output statistical significance for matrices and dataframes Simple and exact expressions

### Usage

```
add_significance_symbols(p_value_table, simple = FALSE)
```

### Arguments

| | |
|---|---|
| p_value_table | A matrix of p-values, or other numerical matrix |
| simple | Whether to represent the * mark simply or precisely with ***. |

### Value

Matrices marked with a *.

### Examples

```
\#\# Not run:

#' p_value <- matrix(c(0, 0.00001, 0.00000,
                   0, 0.00000, 0.01497,
                   0, 0.00013, 0.00000,
                   0, 0.00000, 0.00102),
                 nrow = 4, byrow = TRUE,
                 dimnames = list(c("공명음", "마찰음", "유기음_경음", "평파열음_평파찰음"),
                                 c("H", "H(H)", "L")))
    # Add significance symbols
    # p_value

    add_significance_symbols(p_value)

##                   H     H(H)  L
##공명음             "***" "***" "***"
##마찰음             "***" "***" "*"
##유기음_경음        "***" "***" "***"
##평파열음_평파찰음 "***" "***" "**"




## End(Not run)
```

---

`add_sig_star` *Functions that output statistical significance for matrices and dataframes Simple and exact expressions*

---

### Description

Functions that output statistical significance for matrices and dataframes Simple and exact expressions

### Usage

```
add_sig_star(p_value_table, simple = FALSE)
```

### Arguments

| | |
|---|---|
| `p_value_table` | A matrix of p-values, or other numerical matrix |
| `simple` | Whether to represent the * mark simply or precisely with ***. |

### Value

Matrices marked with a *.

### Examples

```
\#\# Not run:

#' p_value <- matrix(c(0, 0.00001, 0.00000,
                   0, 0.00000, 0.01497,
                   0, 0.00013, 0.00000,
                   0, 0.00000, 0.00102),
                 nrow = 4, byrow = TRUE,
                 dimnames = list(c("공명음", "마찰음", "유기음_경음", "평파열음_평파찰음"),
                                 c("H", "H(H)", "L")))
    # Add significance symbols
    # p_value

    add_significance_symbols(p_value)

##                   H     H(H)  L
##공명음             "***" "***" "***"
##마찰음             "***" "***" "*"
##유기음_경음        "***" "***" "***"
##평파열음_평파찰음 "***" "***" "**"




## End(Not run)
```

---

`add_stat` 데이터에서변수를택하면함수에맞는새로운열을추가하는함수

---

### Description

데이터에서변수를택하면함수에맞는새로운열을추가하는함수

### Usage

```
add_stat(
  data,
  ...,
  fun = mean,
  col_name = NULL,
  na.rm = TRUE,
  margin = 1,
  type = "data"
)
```

### Arguments

| | |
|---|---|
| data | data.frame |
| ... | cols |
| fun | mean, ad, .... |
| col_name | New variable |
| na.rm | na.rm=TRUE |
| margin | margin 1, 2 |
| type | data, col 2type |

### Value

data.fram or vector

### Examples

```
\#\# Not run:

df = data.frame(A = c(1, 2, 3, 4, 5), B = c(6, 7, NA, 9, 10), C = c(11, 12, 13, 14, 15) )

df%>% add_stat( "AC_MEAN","A", "C", fun = mean)
df%>% add_stat( "AC_MEAN","A", "C", fun = mean, type="col")
df %>%add_stat( "A", "C", type="col")
df %>%add_stat( "A", "C")
df %>%add_stat( "A", "C", fun = mean, "AC_MEAN")

df %>%add_stat( "A", "C", "AC_MEAN",fun = sd)

df %>%add_stat(col_name = "AC_MEAN","A", "C", fun = mean)
df %>%add_stat("A", "C", fun = mean, col_name = "AC_MEAN")
df %>%add_stat("A", "C", fun = mean)
df %>%add_stat(c("A", "C"))
df %>%add_stat("ROWs", c("A", "C"))

mtcars %>% add_stat("MEAN", "am","vs")
mtcars %>% add_stat("am","vs")
mtcars %>% add_stat("MEAN", c("am","vs"))
mtcars %>% add_stat("MEAN", c("mpg","qsec"))
mtcars %>% add_stat("MEAN", c("mpg","qsec"), col_name="ACME")
df


## End(Not run)
```

---

## add_sum *Functions to sum margins after adding ratios*

---

### Description

Functions to sum margins after adding ratios

### Usage

```
add_sum(
  dataset,
  v1 = "a1",
  v2 = "성조",
  raw = FALSE,
  add_ratio = TRUE,
  plot = FALSE,
  color = TRUE,
  ylab = "onset",
  xlab = "speaker",
  cex = 1.2,
  title = NULL,
  type = "df"
)
```

### Arguments

| | |
|---|---|
| dataset | data(rwa=T)or table |
| v1 | col |
| v2 | row |
| raw | TRUE dataset. not table |
| add_ratio | using add_ration function |
| plot | mosicplot |
| color | color 1:4, etc |
| ylab | yalb |
| xlab | xlab |
| cex | size |
| title | title |
| type | all, df, res |

### Value

matrix

### Examples

```
#'
\#\# Not run:

#' #'mat = matrix(c(36, 67, 11,
          0, 35, 44,
         41, 108, 1,
          56, 87, 54),
           nrow = 4, byrow = TRUE.
   dimnames = list(c("공명음", "마찰음",
     "유기음_경음", "평파열음_평파찰음"),  c("H", "H(H)", "L")))
## step1
  mat %>% add_ratio()
##conbind
```

```
 mat %>% add_ratio_df()

##add margins
 mat %>% add_sum()

 #                          H H(H)+H(H)_1         L SUM
 #공명음             26(11.5%)   35(19.4%) 36(28.1%)  97
 #마찰음             49(21.6%)   62(34.4%) 35(27.3%) 146
 #유기음_경음       108(47.6%)   82(45.6%)  11(8.6%) 201
 #평파열음_평파찰음  44(19.4%)     1(0.6%) 46(35.9%)  91
 #SUM                      227         180       128 535




## End(Not run)
```

---

`add_t_sig` *add_t_sig*

---

## Description

add_t_sig

## Usage

```
add_t_sig(
  data,
  est,
  se,
  col = ncol(data) + 1,
  unite = FALSE,
  digits = 3,
  ns = " ns"
)
```

## Arguments

| | |
|---|---|
| `data` | data |
| `col` | position t value |
| `unite` | unite sig star +t |
| `digits` | 3 |
| `ns` | ns is ns sig |

## Value

data

## Examples

```
\#\# Not run:
#'
New_df = data.frame(rownames = c('IMAG -> EXPE', 'IMAG -> QUAL', 'IMAG -> VAL', 'IMAG -> SAT'),
  Original = c(0.578959128452789, 0.491156739303251, 0.393411171344413, 0.490971137879492),
 Mean.Boot = c(0.584251828061525, 0.495822941940513, 0.400070801457516, 0.5022633230577),
 Std.Error = c(0.0518709126012371, 0.0510037451930923, 0.0528454527326974, 0.0681551639743143),
 perc.025 = c(0.477709787839589, 0.391876781940297, 0.308196019466015, 0.369649328752085),
  perc.975 = c(0.664779330969341, 0.58560953201285, 0.495717252789497, 0.614490783702303)
)
#Auto-create a separate T without creating a SIG and combine it with the SIG to move it
New_df %>% add_t_sig(est=3, se=4, col=4, unite=T)


# Move only sigs after creating a t
New_df %>% add_t_sig(est=3, se=4, col=4, unite=F)

# 열을 결합
New_df %>% add_t_sig(est=3, se=4, col=4, unite=F) %>%Unite(2,4)

New_df %>% add_t_sig(est=3, se=4, col=4, unite=F)%>%Round(3) %>%Unite(2,4)


## End(Not run)
```

---

`all_na_zero` *all_na_zero*

---

## Description

all_na_zero

## Usage

```
all_na_zero(df, imp = 0)
```

## Arguments

| | |
|---|---|
| `df` | data.frame |
| `imp` | implement default = 0 |

## Value

NA to 0

## all_na_zero1 *NA to zero*

### Description

NA to zero

### Usage

```
all_na_zero1(data, fun = "extend")
```

### Arguments

| | |
|---|---|
| data | data.frame |
| fun | 'extend', 'base' are equal method |

### Value

Na data are changed zero

## alpha_apa *alpha_apa report*

### Description

alpha_apa report

### Usage

```
alpha_apa(data)
```

### Arguments

| | |
|---|---|
| data | 분석결과 |

### Value

apa 보고서

### Examples

```
\#\# Not run:

#'
# 진로동기 신뢰도 분석
jjstat::bind_alpha_table(jutr2%>% select(A01:A15),
                         c(1:5), c(6:10), c(11:15)) %>%nice_Var()%>% nice_table()%>%
  replace_df("a","진로정체감") %>%
  replace_df("b","진로통찰력") %>%
  replace_df("c","진로탄력성") %>%
  alpha_apa()


## End(Not run)
```

`alpha_table` *Function to tabulate reliability*

## Description

Function to tabulate reliability

## Usage

```
alpha_table(
  alpha_data_res,
  title = "",
  show = "data",
  format = "markdown",
  digits = 3,
  check.keys = TRUE,
  variable = "."
)
```

## Arguments

| | |
|---|---|
| `alpha_data_res` | alpha result |
| `title` | your title |
| `show` | 'data', 'markdown' |
| `format` | markdown, html |
| `digits` | default 3 |
| `check.keys` | check.keys=TRUE |
| `variable` | . output |

## Examples

```
\#\# Not run:

stat_onl %>% select(S_Review, S_Add_learn, S_Feedback) %>%
   jjstat::alpha_table(show="data", variable = "A" )


bind_rows(
 stat_onl %>% select(S_Review, S_Add_learn, S_Feedback) %>%
   jjstat::alpha_table(show="data", variable = "A" ),
 stat_onl %>% select(SE_Place, SE_Time) %>%
   jjstat::alpha_table(show="data", variable = "B"),
 stat_onl %>% select(On_Satisfy, On_Joy, On_Easy, On_Engage) %>%
   jjstat::alpha_table(show="data", variable = "C"),
 stat_onl %>% select(upgrade,satisfy,fatigue_inverse) %>%
   jjstat::alpha_table(show="data", variable = "D")
) |> markdown_table(digits=3, font_size = 12,
                    caption= "total Cronbach alpha")

## End(Not run)
```

anova_apa *Report aov data (format APA)*

## Description

Report aov data (format APA)

## Usage

```
anova_apa(
  aov_post_data,
  ns = 7,
  np = 6,
  ndf = 4,
  nt = 5,
  digits = 3,
  md = FALSE,
  posthoc_type = "The Multi Least Significant Difference(MLSD)"
)
```

## Arguments

| | |
|---|---|
| aov_post_data | aov_post data |
| ns | ncol sig column 7 |
| np | ncol p value column 6 |
| ndf | ncol df column 4 |
| nt | ncol t value column 5 |
| digits | digits 3 |
| md | md = TRUE is markdown outpput |

## Value

report data

## Examples

```
\#\# Not run:
aov(mpg ~ cyl , data=Mtcars)%>% aov_post() %>% anova_apa()
# This is error
aov(mpg ~ cyl , data=Mtcars)%>%aov_post(type="contrast")%>% anova_apa()

# aov(mpg ~ cyl , data=Mtcars)%>%aov_post(type="all")

aov(mpg ~ cyl , data=Mtcars)%>%aov_post(type="all")%>% anova_apa()
aov(mpg ~ cyl , data=Mtcars)%>%aov_post(type="res")%>% anova_apa()
```

```
## End(Not run)
```

---

`aov_apa` *Report aov data (format APA)*

---

## Description

Report aov data (format APA)

## Usage

```
aov_apa(
  aov_post_data,
  ns = 7,
  np = 6,
  ndf = 4,
  nt = 5,
  digits = 3,
  md = FALSE,
  posthoc_type = "The Multi Least Significant Difference(MLSD)"
)
```

## Arguments

| | |
|---|---|
| `aov_post_data` | aov_post data |
| `ns` | ncol sig column 7 |
| `np` | ncol p value column 6 |
| `ndf` | ncol df column 4 |
| `nt` | ncol t value column 5 |
| `digits` | digits 3 |
| `md` | md = TRUE is markdown outpput |

## Value

report data

## Examples

```
\#\# Not run:
aov(mpg ~ cyl , data=Mtcars)%>%aov_post()%>% aov_apa()
# This is error
aov(mpg ~ cyl , data=Mtcars)%>%aov_post(type="contrast")%>% aov_apa()

# aov(mpg ~ cyl , data=Mtcars)%>%aov_post(type="all")

aov(mpg ~ cyl , data=Mtcars)%>%aov_post(type="all")%>% aov_apa()
aov(mpg ~ cyl , data=Mtcars)%>%aov_post(type="res")%>% aov_apa()


## End(Not run)
```

`aov_df` *#' Generate an AOV results table for multiple independent variables*

## Description

#' Generate an AOV results table for multiple independent variables

## Usage

```
aov_df(
  data,
  dv_var = NULL,
  iv_vars = NULL,
  grp_mean = TRUE,
  unite_p = FALSE,
  unite_F = FALSE,
  digits = 2,
  posthoc = "lsd",
  p.adj = "bonferroni",
  sig = FALSE
)
```

## Arguments

| | |
|---|---|
| `data` | data.frame |
| `dv_var` | dv |
| `iv_vars` | c(iv1, iv2....) |
| `grp_mean` | 'grooup mean remove |
| `unite_p` | pvalue unite |
| `unite_F` | Fvalue unite |
| `posthoc` | 'tukey, scheffe, duncan, lsd |
| `p.adj` | p.adj=c("none","holm","hommel", "hochberg", "bonferroni", "BH", "BY", "fdr") only lsd test |
| `sig` | 'sig = T' is add star |

## Value

table

## Examples

```
\#\# Not run:
aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"))

aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"),
           mean = TRUE)
```

```
aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"), grp_mean = FALSE)
aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"), grp_mean = FALSE) %>%


aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"),
 grp_mean = TRUE)

aov_table(data = mtcars, dv_var = "mpg",
 iv_vars = c("cyl", "gear", "carb"),
 grp_mean = TRUE, unite_F = TRUE)

aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"),
 grp_mean = TRUE, unite_F = TRUE) %>%
  markdown_table()

# park516d데이터 이용
park516d %>%
  select(1:6,teaching, academyType) %>%  #주요변수 선택
  bind_add_stat(c("RV_Teah",1:6)) %>% # 평균을 이용하여 항목묶음
  aov_df(dv_var="RV_Teah",
         iv_vars =c("teaching", "academyType"), sig = T)



## End(Not run)
```

---

`aov_df2` *Perform ANOVA with emmeans and extract group letters and p-values*

---

## Description

This function performs ANOVA for each independent variable (IV) and extracts the estimated marginal means (emmeans), post-hoc comparison group letters, and p-values, based on the specified multiple comparisons adjustment method.

## Usage

```
aov_df2(
  data,
  dv_var,
  iv_vars,
  adjust = "none",
  alpha = 0.05,
  digits = 2,
  show_group_letter = TRUE,
  adjust_only = TRUE
)
```

### Arguments

| | |
|---|---|
| data | A data.frame containing the variables. |
| dv_var | The name of the dependent variable (character). |
| iv_vars | A character vector of independent variable names. |
| adjust | Adjustment method for pairwise comparisons. One of: "none", "tukey", "bonferroni", "holm", "sidak", "fdr", "scheffe". |
| alpha | Significance level (default = 0.05). |
| digits | Number of digits to round numeric results (default = 2). |
| show_group_letter | Logical. Whether to show group letters (default = TRUE). |
| adjust_only | Logical. If TRUE and adjust != "none", only adjusted p-values (adj_p) are shown (default = TRUE). |

### Details

## Adjustment Methods - '"none"': No adjustment for multiple comparisons (raw p-values). Can inflate Type I error. - '"tukey"': Tukey's Honest Significant Difference method. Simultaneously compares all pairwise mean differences. - '"bonferroni"': Conservative method; adjusts p-values by multiplying by the number of comparisons. - '"holm"': Sequential Bonferroni procedure; less conservative and more powerful alternative. - '"sidak"': Adjustment assuming independent comparisons. - '"fdr"': Controls the False Discovery Rate (Benjamini-Hochberg procedure). - '"scheffe"': Not supported in 'emmeans'; typically used for general contrasts. - '"duncan"', '"lsd"': Not supported in 'emmeans'; internally treated as '"none"'.

### Value

A tibble with ANOVA results per IV level, group letters, and p-values.

### Examples

```
\#\# Not run:
# Basic usage
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"))

# Using Bonferroni adjustment
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"),
               adjust = "bonferroni", adjust_only = FALSE)

# Showing only adjusted p-values
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"),
               adjust = "holm", adjust_only = TRUE)

# Tukey fallback warning
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"),
```

```
                adjust = "tukey")

# Unsupported methods fallback
aov_df_emmeans(data = neulbom_p2_stat2,
                dv_var = "relation",
                iv_vars = c("gender", "age", "child"),
                adjust = "duncan")

## End(Not run)
```

---

`aov_df_emmeans` *Perform ANOVA with emmeans and extract group letters and p-values*

---

## Description

This function performs ANOVA for each independent variable (IV) and extracts the estimated marginal means (emmeans), post-hoc comparison group letters, and p-values, based on the specified multiple comparisons adjustment method.

## Usage

```
aov_df_emmeans(
  data,
  dv_var,
  iv_vars,
  adjust = "none",
  alpha = 0.05,
  digits = 2,
  show_group_letter = TRUE,
  adjust_only = TRUE
)
```

## Arguments

| | |
|---|---|
| `data` | A data.frame containing the variables. |
| `dv_var` | The name of the dependent variable (character). |
| `iv_vars` | A character vector of independent variable names. |
| `adjust` | Adjustment method for pairwise comparisons. One of: ”none”, ”tukey”, ”bonferroni”, ”holm”, ”sidak”, ”fdr”, ”scheffe”. |
| `alpha` | Significance level (default = 0.05). |
| `digits` | Number of digits to round numeric results (default = 2). |
| `show_group_letter` | Logical. Whether to show group letters (default = TRUE). |
| `adjust_only` | Logical. If TRUE and adjust != ”none”, only adjusted p-values (adj_p) are shown (default = TRUE). |

### Details

## Adjustment Methods - ‘"none"‘: No adjustment for multiple comparisons (raw p-values). Can inflate Type I error. - ‘"tukey"‘: Tukey's Honest Significant Difference method. Simultaneously compares all pairwise mean differences. - ‘"bonferroni"‘: Conservative method; adjusts p-values by multiplying by the number of comparisons. - ‘"holm"‘: Sequential Bonferroni procedure; less conservative and more powerful alternative. - ‘"sidak"‘: Adjustment assuming independent comparisons. - ‘"fdr"‘: Controls the False Discovery Rate (Benjamini-Hochberg procedure). - ‘"scheffe"‘: Not supported in ‘emmeans‘; typically used for general contrasts. - ‘"duncan"‘, ‘"lsd"‘: Not supported in ‘emmeans‘; internally treated as ‘"none"‘.

### Value

A tibble with ANOVA results per IV level, group letters, and p-values.

### Examples

```
\#\# Not run:
# Basic usage
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"))

# Using Bonferroni adjustment
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"),
               adjust = "bonferroni", adjust_only = FALSE)

# Showing only adjusted p-values
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"),
               adjust = "holm", adjust_only = TRUE)

# Tukey fallback warning
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"),
               adjust = "tukey")

# Unsupported methods fallback
aov_df_emmeans(data = neulbom_p2_stat2,
               dv_var = "relation",
               iv_vars = c("gender", "age", "child"),
               adjust = "duncan")

## End(Not run)
```

---

`aov_post` *ANOVA posthoc and plot, Contrast data*

---

### Description

ANOVA posthoc and plot, Contrast data

### Usage

```
aov_post(aov_data, type = "df", adjust = "none", lsd = FALSE)
```

### Arguments

| | |
|---|---|
| aov_data | aov data |
| type | type is all, res, contras, df, raq(plotdata), g(plot), aov, anova |
| adjust | adjust =”none”, bonferroni, sidak etc |
| lsd | plosthoc LSD |

### Value

contrast and g

### Examples

```
\#\# Not run:
Mtcars <- mtcars%>%as_trt("cyl")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post()
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="all")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="res")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="df")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="call")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="formula")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="g")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="contrast")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="aov")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="anova")
aov(mpg ~ cyl , data=Mtcars)%>% aov_post(type="posthoc", lsd=TRUE)

#report
aov(mpg ~ cyl , data=Mtcars)%>%aov_post()%>% anova_apa()

## End(Not run)
```

---

## aov_post2 *ANOVA posthoc and plot, Contrast data*

---

### Description

Post-hoc analysis and plotting for ANOVA results. Supports various posthoc methods including LSD, Duncan, Scheffe, Tukey HSD, SNK, FDR and Bonferroni correction.

### Usage

```
aov_post2(
  aov_data,
  type = "df",
  adjust = "none",
  posthoc = TRUE,
  posthoc_method = "LSD"
)
```

### Arguments

| | |
|---|---|
| aov_data | An object returned by aov(); ANOVA model object. |
| type | Output type. One of: "all", "res", "contrast", "df", "raw", "g", "aov", "anova", "call", "formula", "posthoc". |
| adjust | Method for p-value adjustment for emmeans contrast. One of "none", "bonferroni", "fdr", etc. Only used in contrast. |
| posthoc | Logical; if TRUE, performs posthoc test using agricolae. |
| posthoc_method | Posthoc method: one of "LSD", "BONFERRONI", "FDR", "SCHEFFE", "DUNCAN", "TUKEY", "SNK". |

### Value

Depending on type, returns a list or specific object.

### Examples

```
\#\# Not run:
library(agricolae)
library(dplyr)
library(emmeans)
library(broom)
library(ggplot2)

# Example 1: mtcars dataset (mpg ~ cyl)
mtcars$cyl <- as.factor(mtcars$cyl)
fit1 <- aov(mpg ~ cyl, data = mtcars)
aov_post(fit1, type = "df", posthoc = TRUE, posthoc_method = "LSD")
aov_post(fit1, type = "df", posthoc = TRUE, posthoc_method = "BONFERRONI")
aov_post(fit1, type = "df", posthoc = TRUE, posthoc_method = "FDR")
aov_post(fit1, type = "df", posthoc = TRUE, posthoc_method = "SCHEFFE")
aov_post(fit1, type = "df", posthoc = TRUE, posthoc_method = "DUNCAN")
aov_post(fit1, type = "df", posthoc = TRUE, posthoc_method = "TUKEY")
aov_post(fit1, type = "df", posthoc = TRUE, posthoc_method = "SNK")

# Example 2: ToothGrowth dataset (len ~ dose)
ToothGrowth$dose <- as.factor(ToothGrowth$dose)
fit2 <- aov(len ~ dose, data = ToothGrowth)
aov_post(fit2, type = "df", posthoc = TRUE, posthoc_method = "DUNCAN")

# Example 3: iris dataset (Sepal.Length ~ Species)
fit3 <- aov(Sepal.Length ~ Species, data = iris)
aov_post(fit3, type = "df", posthoc = TRUE, posthoc_method = "SCHEFFE")

## End(Not run)
```

`aov_rename` *aov_rename*

## Description

aov_rename

aov_rename

## Usage

```
aov_rename(data)

aov_rename(data)
```

## Arguments

| | |
|---|---|
| `data` | data |

## Value

result

result

`aov_table` *Generate an AOV results table for multiple independent variables*

## Description

Generate an AOV results table for multiple independent variables

Generate an AOV results table for multiple independent variables

## Usage

```
aov_table(
  data,
  dv_var = NULL,
  iv_vars = NULL,
  grp_mean = TRUE,
  unite_p = FALSE,
  unite_F = FALSE,
  digits = 2,
  posthoc = "lsd",
  p.adj = "bonferroni",
  sig = FALSE
)

aov_table(
  data,
  dv_var = NULL,
```

```
  iv_vars = NULL,
  grp_mean = TRUE,
  unite_p = FALSE,
  unite_F = FALSE,
  digits = 2,
  posthoc = "lsd",
  p.adj = "bonferroni",
  sig = FALSE
)
```

## Arguments

| | |
|---|---|
| data | data.frame |
| dv_var | dv |
| iv_vars | c(iv1, iv2....) |
| grp_mean | 'grooup mean remove |
| unite_p | pvalue unite |
| unite_F | Fvalue unite |
| posthoc | 'tukey, scheffe, duncan, lsd |
| p.adj | p.adj=c("none","holm","hommel", "hochberg", "bonferroni", "BH", "BY", "fdr") only lsd test |
| sig | 'sig = T' is add star |

## Value

table

table

## Examples

```
\#\# Not run:
mtcars <- mtcars%>%as_trt("cyl", "gear", "carb")
aov_table(data = mtcars, dv_var = "mpg", iv_vars = c("cyl", "gear", "carb"))

aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"), grp_mean = FALSE)
aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"), grp_mean = FALSE) %>%
aov_table_apaOP

aov_table(data = mtcars, dv_var = "mpg",
 iv_vars = c("cyl", "gear", "carb"), grp_mean = TRUE)

aov_table(data = mtcars, dv_var = "mpg",
 iv_vars = c("cyl", "gear", "carb"), grp_mean = TRUE, unite_F = TRUE)

aov_table(data = mtcars, dv_var = "mpg",
 iv_vars = c("cyl", "gear", "carb"),
  grp_mean = TRUE, unite_F = TRUE) %>% aov_rename()
  markdown_table()
```

```
## End(Not run)



\#\# Not run:
mtcars <- mtcars%>%as_trt("cyl", "gear", "carb")
aov_table(data = mtcars, dv_var = "mpg", iv_vars = c("cyl", "gear", "carb"))

aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"), grp_mean = FALSE)
aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"), grp_mean = FALSE) %>%
aov_table_apaOP

aov_table(data = mtcars, dv_var = "mpg",
 iv_vars = c("cyl", "gear", "carb"), grp_mean = TRUE)

aov_table(data = mtcars, dv_var = "mpg",
 iv_vars = c("cyl", "gear", "carb"), grp_mean = TRUE, unite_F = TRUE)

aov_table(data = mtcars, dv_var = "mpg",
 iv_vars = c("cyl", "gear", "carb"),
  grp_mean = TRUE, unite_F = TRUE) %>% aov_rename()
  markdown_table()

## End(Not run)
```

---

`apa_aov_df` *total anova report*

---

## Description

total anova report

## Usage

```
apa_aov_df(data, show = TRUE, dv_var = NULL, iv_vars = NULL)
```

## Arguments

| | |
|---|---|
| `data` | aoa_df(grp_mean=F) |
| `show` | show data |

## Value

report

### Examples

```
\#\# Not run:
aov_table(data = mtcars, dv_var = "mpg",
iv_vars = c("cyl", "gear", "carb"),
grp_mean = FALSE)%>% apa_aov_df()

apa_aov_df(data = mtcars, dv_var = "mpg",
 iv_vars = c("cyl", "gear", "carb"))

## End(Not run)
```

---

## apa_aov_df_res *apa_aov_df_res, aov_df* 의분석결과

---

### Description

apa_aov_df_res, aov_df 의분석결과

### Usage

```
apa_aov_df_res(data, dv_col = 2, iv_col = 1, show = TRUE, digits = 3)
```

### Arguments

| | |
|---|---|
| data | data |
| dv_col | dv |
| iv_col | iv |
| show | TRUE, data result |
| digits | 3 |

### Value

결과해석 text

### Examples

```
\#\# Not run:
#'
# 테스트 데이터로 실행
data <- data.frame(
  독립변수 = c("LCA_class", "", "", "LCA_class",
  "", "", "LCA_class", "", "", "LCA_class", "", ""),
  종속변수 = c("진로동기", "진로동기", "진로동기",
  "진로태도성숙", "진로태도성숙", "진로태도성숙",
  "진로결정자기효능감", "진로결정자기효능감", "진로결정자기효능감",
  "진로준비행동", "진로준비행동", "진로준비행동"),
  df1 = c(2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2),
  df2 = c(511, 511, 511, 511, 511, 511,
   511, 511, 511, 511, 511, 511),
  F = c(132.64, NA, NA, 107.45, NA, NA,
```

```
   138.02, NA, NA, 106.1, NA, NA),
  p = c("4.00665784818538e-47", NA, NA,
   "1.11197564656087e-39", NA, NA,
   "1.18550277068589e-48", NA, NA,
   "2.8829281865653e-39", NA, NA)
)

# show=TRUE로 결과 출력
apa_results <- apa_aov_df_res(data, dv_col = 2, iv_col = 1, show = TRUE)


## End(Not run)
```

---

`aroian_sobel_test_general`

*Aroian Approximation Sobel Test generalized function*

---

## Description

Aroian Approximation Sobel Test generalized function

## Usage

```
aroian_sobel_test_general(
  a,
  SE,
  roburst = FALSE,
  n = NULL,
  k = 2,
  digits = 5,
  conf = 0.95
)
```

## Arguments

| | |
|---|---|
| `a` | est |
| `SE` | se |
| `roburst` | roburst T whide correction |
| `n` | sample size |
| `k` | number of regressin coefs, default 2 |
| `digits` | 5 |
| `conf` | 0.90, 0.95, 0.99, 0.999 |

## Value

result

## Examples

```
\#\# Not run:
#'
# Example usage
a <- 0.5  # Example coefficient a
b <- 0.3  # Example coefficient b
sa <- 0.1 # Standard error of a
sb <- 0.05 # Standard error of b

Sobel_test(a, b, sa, sb)
Sobel_test(a, b, sa, sb, "all")
Sobel_test(a, b, sa, sb, "Sobel")

Sobel_test(a, b, sa, sb,"Aroian")

Sobel_test(a, b, sa, sb,"Goodman")

#'
# 예제 데이터
a <- c(0.5, 0.3, 0.4)
SE <- c(0.1, 0.1, 0.1)

# 함수 실행 (기본 95% 신뢰구간)
aroian_sobel_test_general(a, SE)
print(result_95)

# 함수 실행 (90% 신뢰구간)
result_90 <- aroian_sobel_test_general(a, SE, conf=0.90)
print(result_90)

# 함수 실행 (99% 신뢰구간)
result_99 <- aroian_sobel_test_general(a, SE, conf=0.99)
print(result_99)

# 함수 실행 (99.9% 신뢰구간)
result_999 <- aroian_sobel_test_general(a, SE, conf=0.999)
print(result_999)


aroian_sobel_test_general(a = c(0.2, 0.3, 0.4, 0.5),
                          SE = c(0.05, 0.04, 0.03, 0.02))%>%unite_ci()


## End(Not run)
```

---

`arrange_by_mind` *Arrange dataframe by user-defined priority on rhs column*

---

## Description

This function arranges a data frame according to a user-defined priority of values in the `rhs` column. If no priority is supplied, the data frame is sorted alphabetically by `rhs`.

### Usage

```
arrange_by_mind(df, priority = NULL)
```

### Arguments

| | |
|---|---|
| df | A data frame containing at least a column named rhs. |
| priority | An optional character vector specifying the preferred order of values in the rhs column. If NULL, rows are sorted alphabetically by rhs. |

### Details

When priority is provided, values of rhs that appear in priority are ordered first according to that order. Remaining values are placed afterward and sorted alphabetically. If priority is NULL, the function simply sorts the data frame alphabetically by rhs.

### Value

A data frame sorted by user-defined priority and/or alphabetically by rhs.

### Examples

```
\#\# Not run:
df <- data.frame(
  lhs = c("intention", "access", "efficacy"),
  rhs = c("barrier", "experience", "grade"),
  est = c(0.2, 0.4, -0.1)
)

# Example 1: Alphabetical ordering only
arrange_by_mind(df)

# Example 2: User-defined priority
arrange_by_mind(df,
  priority = c("experience", "grade")
)

# Example 3: Partial priority specification
arrange_by_mind(df,
  priority = "experience"
)

## End(Not run)
```

---

## as_trt *Treat selected variables as factors*

---

### Description

Treat selected variables as factors

### Usage

```
as_trt(data, ..., fun = factor)
```

### Arguments

| | |
|---|---|
| `data` | data.frame |
| `...` | variables |
| `fun` | funtion default as.factor |

### Value

data treatment

### Examples

```
\#\# Not run:
 str(mrcars)

 mtcars1 = mtcars %>% sel_as_trt("cyl", "am","vs", "gear", "carb")
 str(mtcars1)

 #col number
 mtcars%>%as_trt(c(2,8:11))%>%str()
#'
# 데이터 프레임 생성
df <- data.frame(matrix(1:500, ncol=100))
colnames(df) <- paste0("P", sprintf("%02d", 1:100))

# 열 번호 범위로 변환
as_trt(df, 81:91)%>%str()
as_trt(df, "P81","P91")%>%str()
as_trt(df, 81, 92)%>%str()
#'



## End(Not run)
```

---

## as_trt2 *Treat selected variables as factors*

---

### Description

Treat selected variables as factors

### Usage

```
as_trt2(data, ..., fun = factor)
```

### Arguments

| | |
|---|---|
| `data` | data.frame |
| `...` | variables |
| `fun` | funtion default as.factor |

### Value

data treatment

### Examples

```
\#\# Not run:
 str(mrcars)

 mtcars1 = mtcars %>% as_trt("cyl", "am","vs", "gear", "carb")
 str(mtcars1)

 #col number
 mtcars%>%as_trt(c(2,8:11))%>%str()
#'
# 데이터 프레임 생성
df <- data.frame(matrix(1:500, ncol=100))
colnames(df) <- paste0("P", sprintf("%02d", 1:100))

# 열 번호 범위로 변환
as_trt(df, 81:91)%>%str()
as_trt(df, "P81:P91")%>%str()
as_trt(df, "P81","P91")%>%str()
as_trt(df, 81, 92)%>%str()
#'



## End(Not run)
```

---

`auto_coding` *Automated Likert scale conversion functions*

---

### Description

Automated Likert scale conversion functions

### Usage

```
auto_coding(
  data,
  ...,
  range = 1:ncol(data),
  custom = NULL,
  rev = FALSE,
  data.fram = FALSE,
  start_number = 1,
  default = c("전혀 그렇지 않다", "그렇지 않다", "보통이다", "그렇다",
    "매우 그렇다")
)
```

### Arguments

| | |
|---|---|
| data | data.frame, vector |
| ... | coding levels |
| range | data col range#' |
| start_number | start_number = 1 |
| defaultc(" 전혀 | 그렇지않다", " 그렇지않다"," 보통이다", " 그렇다", " 매우그렇다") |

### Value

transforamtion data

### Examples

```
\#\# Not run:
#'

# test
jut1_custom <- c("그렇다", "매우 그렇다", "보통이다",
                 "그렇지 않다", "전혀 그렇지 않다")
jut1_custom
jut1_custom%>%rev()


jut1_custom <- c("그렇다", "매우 그렇다", "보통이다", "그렇지 않다", "전혀 그렇지 않다")

auto_coding(jut1_custom, custom=jut1_custom)
auto_coding(jut1_custom, custom=jut1_custom, rev=TRUE)


# Example data
vec1 <- c("그렇다", "그렇다", "매우 그렇다", "그렇다", "매우 그렇다", "매우 그렇다",
          "매우 그렇다", "그렇다", "그렇다", "그렇지 않다", "그렇다", "그렇다",
          "매우 그렇다", "그렇다", "그렇다", "그렇다", "그렇다", "매우 그렇다",
          "전혀 그렇지 않다", "보통이다", "그렇다", "매우 그렇다", "매우 그렇다",
          "보통이다", "그렇다", "매우 그렇다", "그렇다", "그렇다", "매우 그렇다",
          "매우 그렇다", "그렇다", "그렇다", "그렇다", "그렇다", "보통이다",
          "그렇다", "그렇다", "그렇다", "매우 그렇다", "보통이다", "매우 그렇다",
          "매우 그렇다", "그렇다", "그렇다", "그렇지 않다", "그렇다", "그렇다",
          "그렇다", "보통이다", "그렇다", "그렇다", "보통이다", "그렇지 않다",
          "그렇지 않다", "매우 그렇다", "보통이다", "그렇다", "매우 그렇다",
          "그렇다", "매우 그렇다", "그렇다", "그렇다", "그렇다", "그렇다",
          "그렇다", "매우 그렇다", "그렇다", "보통이다", "그렇다", "그렇다",
          "그렇다", "매우 그렇다", "보통이다", "그렇다", "그렇다", "그렇지 않다",
          "그렇지 않다", "그렇지 않다", "그렇지 않다", "매우 그렇다", "매우 그렇다",
          "그렇지 않다", "그렇다", "전혀 그렇지 않다", "그렇다", "그렇다",
          "그렇다", "보통이다", "그렇다", "그렇지 않다", "그렇다", "보통이다",
          "매우 그렇다")

auto_coding(vec1, custom=jut1_custom, rev=FALSE)
auto_coding(vec1, custom=jut1_custom, rev=TRUE)
```

```
# Example data
data1 <- data.frame(
  V1 = c("그렇다", "그렇다", "매우 그렇다", "그렇다", "매우 그렇다",
         "매우 그렇다", "매우 그렇다", "그렇다", "그렇다", "그렇지 않다",
         "그렇다", "그렇다", "매우 그렇다", "그렇다", "그렇다",
         "그렇다", "그렇다", "매우 그렇다",  "전혀 그렇지 않다", "보통이다",
         "그렇다", "매우 그렇다", "매우 그렇다", "보통이다", "그렇다",
         "매우 그렇다", "그렇다", "그렇다", "매우 그렇다",  "매우 그렇다",
         "그렇다", "그렇다", "그렇다", "그렇다", "보통이다",
         "그렇다", "그렇다", "그렇다", "매우 그렇다", "보통이다",
         "매우 그렇다", "매우 그렇다", "그렇다", "그렇다", "그렇지 않다",
         "그렇다", "그렇다",
         "그렇다", "보통이다", "그렇다", "그렇다", "보통이다", "그렇지 않다",
         "그렇지 않다", "매우 그렇다", "보통이다", "그렇다", "매우 그렇다",
         "그렇다", "매우 그렇다", "그렇다", "그렇다", "그렇다", "그렇다",
         "그렇다", "매우 그렇다", "그렇다", "보통이다", "그렇다", "그렇다",
         "그렇다", "매우 그렇다", "보통이다", "그렇다", "그렇다", "그렇지 않다",
         "그렇지 않다", "그렇지 않다", "그렇지 않다", "매우 그렇다", "매우 그렇다",
         "그렇지 않다", "그렇다", "전혀 그렇지 않다", "그렇다", "그렇다",
         "그렇다", "보통이다", "그렇다", "그렇지 않다", "그렇다", "보통이다",
         "매우 그렇다", "보통이다")
)

auto_coding(data1)
data1

# Example data
data2 <- data.frame(
  V1 = c("그렇다", "그렇다", "매우 그렇다", "그렇다", "매우 그렇다",
         "매우 그렇다", "매우 그렇다", "그렇다", "그렇다", "그렇지 않다",
         "그렇다", "그렇다", "매우 그렇다", "그렇다", "그렇다",
         "그렇다", "그렇다", "매우 그렇다",  "전혀 그렇지 않다", "보통이다",
         "그렇다", "매우 그렇다", "매우 그렇다", "보통이다", "그렇다",
         "매우 그렇다", "그렇다", "그렇다", "매우 그렇다",  "매우 그렇다",
         "그렇다", "그렇다", "그렇다", "그렇다", "보통이다",
         "그렇다", "그렇다", "그렇다", "매우 그렇다", "보통이다",
         "매우 그렇다", "매우 그렇다", "그렇다", "그렇다", "그렇지 않다",
         "그렇다", "그렇다"),
  v2=c( "그렇다", "보통이다", "그렇다", "그렇다", "보통이다", "그렇지 않다",
        "그렇지 않다", "매우 그렇다", "보통이다", "그렇다", "매우 그렇다",
        "그렇다", "매우 그렇다", "그렇다", "그렇다", "그렇다", "그렇다",
        "그렇다", "매우 그렇다", "그렇다", "보통이다", "그렇다", "그렇다",
        "그렇다", "매우 그렇다", "보통이다", "그렇다", "그렇다", "그렇지 않다",
        "그렇지 않다", "그렇지 않다", "그렇지 않다", "매우 그렇다", "매우 그렇다",
        "그렇지 않다", "그렇다", "전혀 그렇지 않다", "그렇다", "그렇다",
        "그렇다", "보통이다", "그렇다", "그렇지 않다", "그렇다", "보통이다",
        "매우 그렇다", "보통이다")
)
data2
auto_coding(data2)
# jutr1 %>% auto_coding( range=91:95, custom=c("아니다.","그렇다."),start_number=0)%>%dall("head")
# jutr1 %>% auto_coding( range="L01:L05", custom=c("아니다.","그렇다."),start_number=0)%>% head(30)

## End(Not run)
```

`ave_candidates_plot` *ave_candidates_plot:* 문항제거전후 *AVE* 비교시각화

## Description

ave_candidates_plot: 문항제거전후 AVE 비교시각화

## Usage

```
ave_candidates_plot(
  df,
  latent = NULL,
  range_focus = TRUE,
  font_size = 4,
  padding = 0.04,
  v_line_show = TRUE
)
```

## Arguments

| | |
|---|---|
| `df` | lavaan 객체또는 ave_raise_candidates() 결과 |
| `latent` | 특정잠재변수만시각화 (선택) |
| `range_focus` | TRUE 면 AVE 변화중심으로 x 축범위조정 |
| `font_size` | 텍스트크기 |
| `padding` | 범위조절값 (기본 0.04) |
| `v_line_show` | 각문항의 ave_current 위치에수직선추가여부 |

## Value

ggplot2 객체

## Examples

```
\#\# Not run:
library(lavaan)
library(ggplot2)
library(dplyr)

# 예시 데이터: HolzingerSwineford1939
model <- '
  visual  =~ x1 + x2 + x3
  textual =~ x4 + x5 + x6
  speed   =~ x7 + x8 + x9
'
fit <- cfa(model, data = lavaan::HolzingerSwineford1939)

# 1. lavaan 객체 입력 시 자동 처리
ave_candidates_plot(fit)

# 2. 특정 잠재변수만 보기
ave_candidates_plot(fit, latent = "visual")
```

```
# 3. AVE 변화량이 두드러진 경우에만 확대해서 보기
ave_candidates_plot(fit, range_focus = TRUE)

# 4. AVE 변화 전후 수직선 제거
ave_candidates_plot(fit, v_line_show = FALSE)

# 5. font 크기 조절
ave_candidates_plot(fit, font_size = 3)

# 6. ave_raise_candidates() 결과를 직접 전달
result <- ave_raise_candidates(fit, print_result = FALSE)
ave_candidates_plot(result)

## End(Not run)
```

---

`ave_raise_candidates` *ave_raise_candidates:* 잠재변수별 *AVE* 향상후보문항탐색

---

### Description

lavaan 모형결과를기반으로각잠재변수별로문항제거시 AVE(Average Variance Extracted)의변화량을계산합니다. 모든잠재변수를자동으로분석하거나, 특정잠재변수만선택할수있습니다.

### Usage

```
ave_raise_candidates(fit, latent = NULL, cutoff = -Inf, print_result = TRUE)
```

### Arguments

| | |
|---|---|
| `fit` | lavaan::cfa 또는 lavaan::sem 결과객체 |
| `latent` | 분석대상잠재변수이름 (문자열벡터). NULL 이면모델내모든잠재변수를자동분석합니다. |
| `cutoff` | AVE 변화량 (delta) 의최소기준. 기본값은 -Inf. |
| `print_result` | 논리값. TRUE 이면잠재변수별요약메시지를출력합니다. |

### Value

tibble

**latent** 잠재변수이름

**Item** 문항이름

**lambda** 표준화요인적재량

**ave_current** 현재 AVE

**ave_if_dropped** 문항제거시 AVE

**delta** AVE 변화량 (제거후 - 제거전)

### Examples

```
\#\# Not run:
library(lavaan)
library(dplyr)
library(purrr)
library(magrittr)

#----------------------------------------
# 1. CFA 모델 적합
#----------------------------------------
model <- '
  Visual  =~ x1 + x2 + x3
  Textual =~ x4 + x5 + x6
  Speed   =~ x7 + x8 + x9
'
fit <- cfa(model, data = HolzingerSwineford1939, std.lv = TRUE)

#----------------------------------------
# 2. 모든 잠재변수 자동 분석
#----------------------------------------
ave_raise_candidates(fit)

#----------------------------------------
# 3. 특정 잠재변수만 분석
#----------------------------------------
ave_raise_candidates(fit, latent = "Visual")

#----------------------------------------
# 4. AVE가 0.02 이상 증가하는 경우만 필터링
#----------------------------------------
ave_raise_candidates(fit, cutoff = 0.02)

#----------------------------------------
# 5. 여러 잠재변수 선택 분석
#----------------------------------------
ave_raise_candidates(fit, latent = c("Visual", "Textual"))

## End(Not run)
```

---

`bartlett_test` *bartlett_test*

---

### Description

bartlett_test

### Usage

```
bartlett_test(
  data,
  formula = NULL,
  Mean = "Mean",
  sd = "SD",
  n = "N",
```

```
  var = "variable"
)
```

### Arguments

| | |
|---|---|
| data | data.frame or mean. sd. n |
| formula | dv ~ iv |
| Mean | mean selection |
| sd | sd selection |
| n | n selection |
| var | variable |

### Value

result

### Examples

```
\#\# Not run:

set.seed(20240418)
my_data <- data.frame(
  value = c(group1 <- rnorm(20, mean = 10, sd = 2),
            group2 <- rnorm(20, mean = 12, sd = 2),
            group3 <- rnorm(20, mean = 15, sd = 2)),
  group = rep(c("Group1", "Group2", "Group3"), each = 20)
)
my_data %>%mysummaryBy(value ~ group)%>%select(1:4) %>%bartlett_test()

data = data.frame(row.names = c("1", "2", "3"),
                  var = c('Group1', 'Group2', 'Group3'),
                  Mean = c('10.0782306287367', '11.5824600813951', '14.764623393546'),
                  SD = c('1.88743751774179', '1.9951475292551', '2.07862332265816'),
                  N = c('20', '20', '20'))
data
bartlett_test(data)

# original test
bartlett.test(value ~ group, data=my_data)
#now test
bartlett_test(value ~ group, data=my_data)


#origin
bartlett.test(mpg~cyl, data=mtcars)
#aggregated data
mtcars%>%mysummaryBy(mpg~cyl) %>%bartlett_test()
# Using function
bartlett_test(mpg~cyl, data=mtcars)
#'

## End(Not run)
```

`binary_recode` *binary_recode*

## Description

binary_recode

## Usage

```
binary_recode(data, variable, recode_list, new_name = NULL, tail = "_grp")
```

## Arguments

| | |
|---|---|
| `data` | data |
| `variable` | original variable |
| `recode_list` | list recode variable |
| `new_name` | new name |
| `tail` | tails _grp |

## Value

data

## Examples

```
\#\# Not run:
#'
# 예제 데이터
data <- data.frame(
  학력 = factor(c("고졸", "대학졸", "석박사졸", "고졸", "석박사졸")),
  grade = 1:5
)
data
binary_recode(data, grade, list(미숙자 = 1:3, 숙달자 = 4:5))
binary_recode(data, grade, list(미숙자 = c(1,2,3), 숙달자 = 4:5))
binary_recode(data, "grade", list(미숙자 = 1:3, 숙달자= 4:5), new_name="수준")

binary_recode(data, 학력, list(저학년= 1:2, 고학년= 3))
binary_recode(data, "학력", list(저학년= 1:2, 고학년= 3))
binary_recode(data, "학력", list(저학년= c("고졸","대학졸"), 고학년= 3))
binary_recode(data, 학력, list(저학년= c("고졸","대학졸"), 고학년= 3))
binary_recode(data, 학력, list(저학년= c("고졸","대학졸"), 고학년= "석박사졸"))


## End(Not run)
```

`binary_recode_sep` *Multi-recoding binary variables*

## Description

Multi-recoding binary variables

## Usage

```
binary_recode_sep(data, ..., new_names = NULL, tail = "_grp")
```

## Arguments

| | |
|---|---|
| `data` | data |
| `...` | recode list |
| `new_names` | newnaem list |
| `tail` | tail _grp |

## Value

data

## Examples

```
\#\# Not run:



# 예제 데이터
data <- data.frame(
  학력 = factor(c("고졸", "대학졸", "석박사졸", "고졸", "석박사졸")),
  grade = 1:5
)
data


# 함수 호출
binary_recode_sep(  data,
                    "학력", list(저학년 = 1:2, 고학년 = 3),
                    "grade", list(저학년 = 1:2, 고학년 = 3:5),
                    new_names = c("학력_grp", "grade_grp")
)




## End(Not run)
```

`bind_alpha_table` *Alpha Table Combination*

## Description

Alpha Table Combination

## Usage

```
bind_alpha_table(data, ..., check.keys = TRUE)
```

## Arguments

| | |
|---|---|
| `data` | Data frame containing the variables |
| `...` | Variable names to be included in the alpha table, column name and number. Variable names to be included in the alpha table |
| `check.keys` | check.keys=TRUE |

## Value

alpha table

## Examples

```
\#\# Not run:
## select variable
bind_alpha_table(data = stat_onl, c("S_Review", "S_Add_learn", "S_Feedback"))

#error
bind_alpha_table(data = stat_onl, c(S_Review, S_Add_learn, S_Feedback))

#select col number
bind_alpha_table(data = stat_onl, c(9:11))

# mutilple data
bind_alpha_table( stat_onl,
                  c("S_Review", "S_Add_learn", "S_Feedback"),
                  c("SE_Place", "SE_Time"),
                  c("On_Satisfy", "On_Joy", "On_Easy", "On_Engage"),
                  c("upgrade","satisfy","fatigue_inverse") )


#mixed multiple
bind_alpha_table( stat_onl,
                  c("S_Review", "S_Add_learn", "S_Feedback"),
                  c("SE_Place", "SE_Time"),
                  c("On_Satisfy", "On_Joy", "On_Easy", "On_Engage"),
                  c(5:6, 21) )
```

```
## End(Not run)
```

---

`bind_compare_data` *Table-building functions that pairwise compare two pieces of data*

---

## Description

Table-building functions that pairwise compare two pieces of data

## Usage

```
bind_compare_data(dff1, dff2, digits = 3, pair = TRUE)
```

## Arguments

| | |
|---|---|
| `dff1` | data1 |
| `dff2` | data2 |
| `digits` | rounding |
| `pair` | pari option default TRUE |

## Value

table

---

`bind_lme_model` *Tbind_lme_model*

---

## Description

Tbind_lme_model

## Usage

```
bind_lme_model(
  ...,
  model_name = paste("모형", 1:length(model)),
  title = "모형의 비교"
)
```

## Arguments

| | |
|---|---|
| `...` | input |
| `model_name` | model name |
| `title` | model title |

## Value

table

### Examples

```
\#\# Not run:
data(birthwt)
str(birthwt)
fitm <- lm(bwt ~ age+lwt+factor(race)+smoke+ptl+ht+ui, data=birthwt)
fitm2 <- lm(bwt ~ lwt+factor(race)+smoke+ht+ui, data=birthwt)
bind_lme_model(fitm, fitm2)


## End(Not run)
```

---

`bind_mysummaryBy` *bind_mysummaryBy is group mean of mean*

---

### Description

bind_mysummaryBy is group mean of mean

### Usage

```
bind_mysummaryBy(data, ..., unite = FALSE)
```

### Arguments

| | |
|---|---|
| `data` | data.frame |
| `...` | fomula data ... |
| `unite` | ... bind persional data and group data TRUE |

### Value

Use group data obtained from group descriptive statistics to obtain group-level descriptive statistics (means, standard deviations, etc.) and to analyze a mixed model

### Examples

```
\#\# Not run:
##
bind_mysummaryBy(mtcars, mpg ~ am)

##compare
mysummaryBy(data=mtcars, mpg ~ am)

group_by(mtcars, am) %>%  summarise(mpg = mean(mpg))

 ## A tibble: 2 × 2
 ##    am   mpg
 ##<dbl> <dbl>
 ## 1     0  17.1
 ##2     1  24.4
```

```
bind_mysummaryBy(mtcars, mpg ~ am, mpg ~ vs)


## grp   dv        N  MEAN    SD   MIN   MAX
##  <chr> <chr> <int> <dbl> <dbl> <dbl> <dbl>
##1 am    mpg       2  20.8  5.12  17.1  24.4
##2 vs    mpg       2  20.6  5.61  16.6  24.6
\#

bind_mysummaryBy(mtcars, mpg ~ am, mpg ~ vs, mpg ~cyl)

# A tibble: 3 × 9
# grp   dv        N  MEAN    SD   MIN   MAX      Skew  Kurt
# <chr> <chr> <int> <dbl> <dbl> <dbl> <dbl>     <dbl> <dbl>
#   1 am    mpg       2  20.8  5.12  17.1  24.4 -5.28e-16 -2.75
# 2 vs    mpg       2  20.6  5.61  16.6  24.6  0        -2.75
# 3 cyl   mpg       3  20.5  5.82  15.1  26.7  1.28e- 1 -2.33


jjstat::bind_mysummaryBy(mtcars, mpg ~ am, mpg ~ vs, mpg ~ cyl, unite=TRUE)

# A tibble: 3 × 8
# var         N  MEAN    SD   MIN   MAX      Skew  Kurt
# <chr>   <int> <dbl> <dbl> <dbl> <dbl>     <dbl> <dbl>
#   1 am_mpg      2  20.8  5.12  17.1  24.4 -5.28e-16 -2.75
# 2 vs_mpg      2  20.6  5.61  16.6  24.6  0        -2.75
# 3 cyl_mpg     3  20.5  5.82  15.1  26.7  1.28e- 1 -2.33




## End(Not run)
```

---

`box_cox` *Apply Box-Cox Style Transformations to a Data Frame*

---

## Description

This function applies various mathematical transformations to all numeric columns in a data frame.

## Usage

```
box_cox(df, use = "log")
```

## Arguments

| | |
|---|---|
| `df` | A data frame with numeric columns to be transformed. |
| `use` | A character string specifying the transformation to apply. Options are:<br>• `"log"`: Logarithmic transformation ($\log(x+1)$)<br>• `"sqrt"`: Square root transformation ($\sqrt{x}$)<br>• `"inverse"`: Inverse transformation ($1/(x+1)$) |

- `"square"`: Square transformation ($x^2$)
- `"cube_root"`: Cube root transformation ($x^{1/3}$)
- `"exp"`: Exponential transformation ($e^x$)

### Value

A data frame with transformed numeric columns.

### Examples

```
\#\# Not run:
# Example: Apply transformations to a data frame
df <- data.frame(
  col1 = c(1, 2, 3),
  col2 = c(4, 5, 6)
)

# Logarithmic transformation
box_cox(df, use = "log")

# Square root transformation
box_cox(df, use = "sqrt")

# Cube root transformation
box_cox(df, use = "cube_root")

## End(Not run)
```

---

## `breslowdata_test` *Breslow-Day Test for Homogeneity of the Odds Ratios*

---

### Description

Breslow-Day Test for Homogeneity of the Odds Ratios

### Usage

```
breslowdata_test(data, OR = NA, correct = FALSE, tidy = TRUE)
```

### Arguments

| | |
|---|---|
| `data` | 2x2xk table |
| `OR` | the odds ratio to be tested against. If left undefined (default) the Mantel-Haenszel estimate will be used. |
| `correct` | If TRUE, the Breslow-Day test with Tarone's adjustment is computed, which subtracts an adjustment factor to make the resulting statistic asymptotically chi-square. |
| `tidy` | broom::tidy |

### Details

For the Breslow-Day test to be valid, the sample size should be relatively large in each stratum, and at least 80 Alternatively, it might be better to cast the entire inference problem into the setting of a logistic regression model. Here, the underlying question of the Breslow-Day test can be answered by investigating whether an interaction term with the strata variable is necessary (e.g. using a likelihood ratio test using the anova function).

### Value

result

### Examples

```
\#\# Not run:
#'
migraine <- xtabs(freq ~ .,
                  cbind(expand.grid(treatment=c("active", "placebo"),
                                    response =c("better", "same"),
                                    gender   =c("female", "male")),
                        freq=c(16, 5, 11, 20, 12, 7, 16, 19))
)
# get rid of gender
tab <- xtabs(Freq ~ treatment + response, migraine)
DescTools::Desc(tab)
# only the women
female <- migraine[,, 1]
DescTools::Desc(female)
# .. and the men
male <- migraine[,, 2]
DescTools::Desc(male)
breslowdata_test(migraine)
breslowdata_test(migraine, correct = TRUE)


## End(Not run)
```

---

## calculate_chi_sig *Create oserved/expected tapes after chisq calculation and give them individual significance*

---

### Description

Create oserved/expected tapes after chisq calculation and give them individual significance

### Usage

```
calculate_chi_sig(observed, type = "data", digits = 3, exact = TRUE)
```

### Arguments

| | |
|---|---|
| observed | contigency table |
| type | data, Res(all), chisq_test, cramers v, observed_over_expected_ratios,cell_p_values, p_sig, data, data2 |

| | |
|---|---|
| `digits` | digits= 3 |
| `exact` | yYates's continuity correction |

### Value

matrix, data.frame

### Examples

```
\#\# Not run:
# # calculate_chi_sig(data)
# Example data
matrix(c(36, 67, 11,
          0, 35, 44,
         41, 108, 1,
          56, 87, 54),
           nrow = 4, byrow = TRUE,
   dimnames = list(c("공명음", "마찰음",
     "유기음_경음", "평파열음_평파찰음"),  c("H", "H(H)", "L"))) %>%
     calculate_chi_sig("p_sig")


     matrix(c(36, 67, 11,
              40, 35, 44,
              41, 108, 1,
              56, 87, 54), nrow = 4, byrow = TRUE,
                     dimnames = list(c("공명음", "마찰음",
                                       "유기음_경음", "평파열음_평파찰음"),
                                     c("H", "H(H)", "L"))) %>%
               calculate_chi_sig(simple = TRUE)



  calculate_chi_sig(matrix(c(36, 67, 11,
                      40, 35, 44,
                  41, 108, 1,
              56, 87, 54), nrow = 4, byrow = TRUE,
               dimnames = list(c("공명음", "마찰음",
              "유기음_경음", "평파열음_평파찰음"),
              c("H", "H(H)", "L")))   )


 # calculate_chi_sig(data)

## End(Not run)
```

---

`cal_vif` *calculation vif*

---

### Description

calculation vif

### Usage

```
cal_vif(model)
```

### Arguments

| | |
|---|---|
| model | lm model data |

### Value

vif

### Examples

```
\#\# Not run:
#'
# Create the linear model
model <- lm(hp ~ wt + disp + drat, data = mtcars)

# Calculate VIF using the custom function
cal_vif(model)

lm(mpg ~ wt + disp + drat + hp , data = mtcars) %>%cal_vif()



## End(Not run)
```

---

## centering *centering each vector*

---

### Description

centering each vector

### Usage

```
centering(variable, na.rm = TRUE)
```

### Arguments

| | |
|---|---|
| variable | vector variable in data |
| na.rm | missing data chedk |

`cfa2` *Create CFA result summary by lavaan sem estimated result By Park Joonghee Ph.D*

## Description

Create CFA result summary by lavaan sem estimated result By Park Joonghee Ph.D

## Usage

```
cfa2(
  x,
  format = "markdown",
  htmt_cut = 0.9,
  htmt2 = FALSE,
  cut = 0.7,
  angle = 90,
  cex = 11,
  hjust = 0.9,
  val.size = 4,
  dis.sort = TRUE,
  rename = FALSE,
  var_name = NA,
  digits = 3,
  type = "all"
)
```

## Arguments

| | |
|---|---|
| `x` | lavaan result |
| `format` | knitr markdown decision |
| `htmt_cut` | htmt_cut is default =1. if you want roburst option htmt_cut =0.9 |
| `htmt2` | TRUE using htmt2 |
| `cut` | loading criteran |
| `angle` | tesxt angle |
| `cex` | text size |
| `hjust` | on text value gap |
| `val.size` | text size |
| `dis.sort` | item sort |
| `rename` | item validiation new name apply |
| `var_name` | new text name default NULL |
| `digits` | value rounding |
| `dataset` | this is import name that dataset column name |
| `model` | lavaan model need indices validation name |
| `res` | result type |

## Examples

```
\#\# Not run:
library(lavaan)
data(PoliticalDemocracy)

model1 <- "
  ind60 =~ x1 + x2 + x3
  dem60 =~ y1 + y2 + y3 + y4
  dem65 =~ y5 + y6 + y7 + y8
  dem60 ~ ind60
  dem65 ~ ind60 + dem60
"

fit1 <- sem(model1, data = PoliticalDemocracy)

# 기본 사용법: 전체 결과 출력
cfa2(fit1)

# 특정 결과(res)만 출력 예시
cfa2(fit1, res = "modelfit")        # 모델 적합도
cfa2(fit1, res = "loadings")        # 요인적재량 테이블
cfa2(fit1, res = "alpha")           # 신뢰도(Cronbach alpha)
cfa2(fit1, res = "CR_AVE")          # 수렴타당도(CR, AVE)
cfa2(fit1, res = "Discriminant")    # 판별타당도 (Fornell-Larcker 기준)
cfa2(fit1, res = "htmt", dataset = PoliticalDemocracy, model = model1)  # HTMT 지표
cfa2(fit1, res = "str_cor")         # 잠재변수 간 경로 유의성

# 시각화 결과: 요인적재량 바그래프 출력
cfa2(fit1, res = "loadings_bar")

# 사용자 정의 변수명 적용(rename + var_name)
new_names <- c("Indicator1", "Indicator2", "Indicator3",
               "Democracy1", "Democracy2", "Democracy3", "Democracy4",
               "Democracy5", "Democracy6", "Democracy7", "Democracy8")

# 그래프의 색을 바꾸고 싶은 경우
cfa2(fit, type = "loadings_bar") +
scale_fill_grey()
# rename = TRUE일 경우에만 var_name이 반영됨
cfa2(fit1, rename = TRUE, var_name = new_names)

# rename = FALSE일 경우 var_name은 무시됨
cfa2(fit1, rename = FALSE, var_name = new_names)

# 다양한 출력 포맷 지정
cfa2(fit1, format = "markdown")
cfa2(fit1, format = "html") %>% kableExtra::kable_minimal(full_width = FALSE)

# 모델 간 비교
model2 <- "
  ind60 =~ x1 + x2 + x3
  dem60 =~ y1 + y2 + y3 + y4
  dem65 =~ y5 + y6 + y7 + y8
  dem60 ~ ind60
  dem65 ~ ind60 + dem60
"
```

```
fit2 <- sem(model2, data = PoliticalDemocracy)
CompareFit_diff(fit1, fit2)

# AVE 직접 확인
AVE(fit1)
AVE(fit2)

# 통합 요약 테이블 출력 (CR, AVE 포함)
library(dplyr)
full_join(
  cfa2(fit1, type = "item") %>% dplyr::select(-Accept, -p),
  cfa2(fit1, type = "Convergent") %>% row2col("Latent") %>%
    dplyr::select(-Cronbach) %>% Round(3),
  by = "Latent"
) %>% nice_table()

## End(Not run)
```

---

`cfa_ml` *Multilevel CFA summary wrapper*

---

## Description

다층구조방정식 (CFA/SEM) 모형의 within / between 수준결과를한번에요약하여제공하는래퍼함수이다.

## Usage

```
cfa_ml(
  fit,
  level = c("within", "between"),
  type = c("all", "model_fit", "indicator_validity", "CR_AVE", "item_CR_AVE", "fornell",
    "htmt"),
  digits = 3
)
```

## Arguments

| | |
|---|---|
| `fit` | lavaan multilevel SEM object |
| `level` | character. ”within” 또는”between” |
| `type` | character. 출력제어옵션<br>• ”all” 모든결과를 list 로반환<br>• ”model_fit” 모형적합도<br>• ”indicator_validity” 요인부하량및지표타당도<br>• ”CR_AVE” 잠재변수수준 CR / AVE<br>• ”item_CR_AVE” 문항수준 CR / AVE<br>• ”fornell”Fornell–Larcker 판별타당도<br>• ”htmt”HTMT 판별타당도 |
| `digits` | numeric. 소수점자리수 |

### Value

data.frame 또는 list

### Examples

```
\#\# Not run:
res_all <- cfa_ml(fit_ml, level = "within", type = "all")
res_fit <- cfa_ml(fit_ml, level = "between", type = "model_fit")

## End(Not run)
```

---

## change_node_name *Change node names in semPlot / qgraph object*

---

### Description

semPlot::semPaths() 로생성된구조방정식시각화객체에서노드이름을사용자가지정한 (원래이름, 변경라벨) 쌍에따라일괄적으로변경한다. 본함수는 pipe(

### Usage

```
change_node_name(plot_obj, var_named)
```

### Arguments

| | |
|---|---|
| plot_obj | semPlot 또는 qgraph 객체 |
| var_named | character vector. c(old1, new1, old2, new2, ...) |

### Value

노드명칭이변경된 semPlot 객체

### Examples

```
\#\# Not run:

#################################
# 1. SCCT 구조방정식 모형 정의
#################################

model_scct_indirect <- '

REFF =~ efficacy2 + efficacy3 + efficacy4
OEXP =~ Outcome1 + Outcome2 + Outcome3 + Outcome4 + Outcome5 + Outcome8
INTR =~ Interest1 + Interest2 + Interest3 + Interest4

REFF ~ e1*grade + e2*experience0
OEXP ~ a1*REFF + d1*grade + d2*experience0
INTR ~ b1*REFF + b2*OEXP

intention ~ c1*INTR + c2*OEXP + c3*REFF

grade ~~ experience0
```

```
# 간접효과
ind_grade_REFF        := e1 * c3
ind_grade_REFF_INTR   := e1 * b1 * c1
ind_grade_REFF_OEXP   := e1 * a1 * c2
ind_grade_REFF_OEXP_INTR := e1 * a1 * b2 * c1

ind_exp_REFF          := e2 * c3
ind_exp_REFF_INTR     := e2 * b1 * c1
ind_exp_REFF_OEXP     := e2 * a1 * c2
ind_exp_REFF_OEXP_INTR := e2 * a1 * b2 * c1
'

################################
# 2. 모형 추정
################################

library(lavaan)
fit_scct_indirect <- sem(
  model = model_scct_indirect,
  data  = shindata,
  se = "bootstrap",
  bootstrap = 2000
)

################################
# 3. 노드 명칭 매핑 벡터
################################

var_named1 <- c(
  "REFF", "연구효능감",
  "efficacy2","연구효능2",
  "efficacy3","연구효능3",
  "efficacy4","연구효능4",

  "INTR", "진로흥미",
  "Interest1","진로흥미1",
  "Interest2","진로흥미2",
  "Interest3","진로흥미3",
  "Interest4","진로흥미4",

  "OEXP", "결과기대",
  "Outcome1","결과기대1",
  "Outcome2","결과기대2",
  "Outcome3","결과기대3",
  "Outcome4","결과기대4",
  "Outcome5","결과기대5",
  "Outcome8","결과기대8",

  "grade","학년",
  "experience0","연구경험",
  "intention","진로의지"
)

################################
# 4. SCCT 전용 layout 행렬
################################
```

```
layout_scct <- matrix(
  c(
    NA, "efficacy2", "efficacy3", "efficacy4", NA,
        "Interest1", "Interest2", "Interest3", "Interest4", NA,

    "experience0", NA, "REFF", NA, NA,
        NA, "INTR", NA, NA, NA,

    "grade", NA, "OEXP", NA, NA,
        NA, NA, "intention", NA, NA,

    NA, "Outcome1", "Outcome2", "Outcome3", "Outcome4",
        "Outcome5", "Outcome8", NA, NA, NA
  ),
  nrow = 4,
  byrow = TRUE
)

################################
# 5. 구조방정식 그림 + 노드명 변경
################################

library(semPlot)
library(semptools)

semPlot::semPaths(
  fit_scct_indirect,
  layout        = layout_scct,
  whatLabels    = "std",
  style         = "ram",
  residuals     = FALSE,
  intercepts    = FALSE,
  nCharNodes    = 0,
  sizeLat       = 9,
  sizeMan       = 9,
  edge.label.cex = 0.85,
  edge.label.position = 0.65
) %>%
  semptools::mark_sig(fit_scct_indirect) %>%
  change_node_name(var_named1) %>%  ##### 이것을 이용하여 이름 변경
  plot()


## End(Not run)
```

---

Char2num *Functions to convert character to numeric*

---

## Description

Functions to convert character to numeric

## Usage

```
Char2num(data, iter = NULL, char_col = 1, last_col = FALSE)
```

### Arguments

| | |
|---|---|
| `data` | data.frame |
| `iter` | Select the range of variables to convert |
| `char_col` | Specify which columns should be kept as char col |

### Value

data.frame

### Examples

```
\#\# Not run:

## Replacing a character variable with numeric
mysummary(sats, "sux1") %>%
    mutate(across(everything(), ~ format(., digits = 4))) %>%
    tibble() %>% Char2num()


## Set remove=TRUE if you only want to process the selected variable and output only that.
  mysummary(sats, "sux1") %>%
  mutate(across(everything(), ~ format(., digits = 5))) %>%
    tibble() %>%Char2num(auto=FALSE, iter= 4:ncol(.), remove= TRUE)

##
mysummary(sats, "sux1") %>%
  mutate(across(everything(), ~ format(., digits = 5))) %>%
    tibble() %>%Char2num(auto=FALSE, iter= 4:ncol(.), remove= FALSE)
## same
  mysummary(sats, "sux1") %>%
     mutate(across(everything(), ~ format(., digits = 5))) %>%
     tibble() %>%Char2num(auto=FALSE, iter= 4:ncol(.))


#새로운 데이터
reg_text <- "var B se b t
성취감 -.03 .09 -.03 -.33
인정감 .24 .09 .20 2.50
책임감 .30 .08 .23 3.56***
발전성 -.08 .06 -.10 -1.29
감독 .06 .05 .08 1.35
보수 -.14 .07 -.18 -2.01*
대인관계 .06 .06 .06 .89
작업환경 .33 .09 .34 3.82***
안정성 .17 .05 .26 3.42**
성별 -.05 .09 -.03 -.52
연령 -.20 .07 -.22 -3.01**
학력 .05 .07 .04 .70
소득 .00 .04 .01 .12
종사기간 .17 .06 .23 2.71**
기관규모 -.10 .03 -.20 -3.36**"

# 함수 호출
text2df(reg_text) %>% tibble()
text2df(reg_text)%>% Char2num() %>% tibble()
```

```
text2df(reg_text)%>% Char2num(iter=2:4) %>% tibble()

## End(Not run)
```

---

`Char2num_fct` *fact varaible to num in data.frame*

---

## Description

fact varaible to num in data.frame

## Usage

```
Char2num_fct(df, col = 1)
```

## Arguments

| | |
|---|---|
| `df` | df |
| `col` | select col |

## Value

df

---

`chisq.multcomp` *RVAideMemoire::chisq.multcomp, Pairwise comparisons after a chi-squared goodness-of-fit test*

---

## Description

RVAideMemoire::chisq.multcomp, Pairwise comparisons after a chi-squared goodness-of-fit test

## Usage

```
chisq.multcomp(x, p.method = "fdr")
```

## Arguments

| | |
|---|---|
| `x` | numeric vector (counts). |
| `p.method` | method for p-values correction. See help of p.adjust. |

## Value

result

### Examples

```
\#\# Not run:
#'
counts <- c(49,30,63,59)
chisq.test(counts)
chisq.multcomp(counts)
#comapre
chisq_multcomp(counts)


## End(Not run)
```

---

`chisqTest` *chisq_test 2way*

---

### Description

chisq_test 2way

### Usage

```
chisqTest(Obs, correct = FALSE, digits = 3, cat = TRUE)
```

### Arguments

| | |
|---|---|
| `Obs` | observation |
| `correct` | Yates' continuity correction |
| `digits` | 3 |
| `cat` | chisq.test form |

### Value

chisq

### Examples

```
\#\# Not run:

table(mtcars$am, mtcars$vs) %>% chisq_test()


table(mtcars$am, mtcars$vs) %>% chisq_test(correct=F)

## compare original chisq.test
table(mtcars$am, mtcars$vs) %>% chisq.test()


##Pearson's Chi-squared test with Yates' continuity correction
table(mtcars$am, mtcars$vs) %>% chisq_test(correct=T)

## End(Not run)
```

## chisq_apa 카이제곱 *apa* 보고

### Description

카이제곱 apa 보고

### Usage

```
chisq_apa(data, chi_type = NULL, show = TRUE)
```

### Arguments

| | |
|---|---|
| data | chisq.test result data |
| chi_type | " 적합성검정", " 독립성검정", " 동질성검정" |
| show | TRUE cat output |

### Value

data result

### Examples

```
\#\# Not run:
#'
dfa= matrix(c(22, 26, 55,13, 31, 33,13, 16, 38), ncol=3, byrow=TRUE)
colnames(dfa)=c("가","나","다")
rownames(dfa)=c("A","B","C")
dfa
dfat= t(dfa)
dfat%>%chisq.test()%>%chisq_apa()

## End(Not run)
```

## chisq_contrast *Chisq test Contrast test*

### Description

Chisq test Contrast test

### Usage

```
chisq_contrast(
  data,
  ...,
  test = chisq.test,
  p.adjust = "bonferroni",
  adj = FALSE,
  name = FALSE
)
```

### Arguments

| | |
|---|---|
| `data` | data |
| `...` | contrast |
| `test` | chisq.test, fisher.test |
| `p.adjust` | c("bonferroni","fdr","BH","BY","holm","hochberg","hommel") |
| `adj` | cadjust p value TRUE |
| `name` | rowname, when pairwise |

### Value

data

### Examples

```
\#\# Not run:
#'
d <- matrix(c(584, 24, 1721, 56, 2400, 90, 8164, 289),
            nrow = 4, byrow = TRUE,
            dimnames = list(c("kangnam", "Moksdong", "nowon", "dongtan"), c("Yes", "No")))

d%>%chisq.test()
chisq_contrast(d)
chisq_contrast(d, c(1,2,3,4))


chisq_contrast(d, c(1,1,2,3),c(2,1,2,2),c(2,2,1,2),c(2,2,2,1))
# fisher.test
chisq_contrast(d, c(1,2,2,2),c(2,1,2,2),c(2,2,1,2),c(2,2,2,1), test=fisher.test)
# fisher.exact test
chisq_contrast(d, c(1,2,2,3),c(2,1,2,2),c(2,2,1,2),c(2,2,2,1), test=fisher.test)

chisq_contrast(d, c(1,2,2,2),c(2,1,1,2),c(2,2,1,2),c(2,2,2,1), test=fisher.test)
chisq_contrast(d, c(1,2,2,2),c(2,3,1,2),c(2,2,1,2),c(2,2,2,1), test=fisher.test)
chisq_contrast(d, c(1,2,2,2),c(2,3,1,2),c(2,2,1,3),c(2,2,2,1), test=fisher.test)
chisq_contrast(d, c(1,2,2,2),c(2,3,1,2),c(2,2,1,3),c(3,4,2,1), test=fisher.test)


#other data
data_text = "Tobacco_Dependence (0,5] (5,10] (10,15] (15,20] (20,98]
No_Nicotine_Dependence 130 210 43 114 20
Nicotine_Dependence 119 267 91 254 67"

#method2
data_text%>% text2df(header=T) %>% make_df_text()
nt
nt = data_text%>% text2df(header=T)%>% jjstat::col2row()
nt %>%tibble()
nt=Char2num(nt,1:5)
nt%>%t() %>%as.data.frame()%>%
  chisq_contrast(c(1,2,2,2,2),
                 c(2,1,2,2,2),
                 c(2,2,1,2,2),
                 c(2,3,4,1,2))
```

```
## End(Not run)
```

---

`chisq_gof_posthoc` *chisq_gof_posthoc*

---

## Description

chisq_gof_posthoc

## Usage

```
chisq_gof_posthoc(vector, type = "res", method = "bonferroni", names = TRUE)
```

## Arguments

| | |
|---|---|
| `vector` | data |
| `type` | all, res, p, chisq |
| `method` | p.adust fdr, bonferroni, ... |
| `names` | TRUE using names |

## Value

data

## Examples

```
\#\# Not run:
#'

X <- c(49, 30, 63, 59)

chisq_gof_posthoc(X, "res")
chisq_gof_posthoc(X,"all")$gof_test
chisq_gof_posthoc(X,"all")$p
chisq_gof_posthoc(X,"all")$chisq
chisq_gof_posthoc(X, typ="all")
chisq_gof_posthoc(X, typ="res")
chisq_gof_posthoc(X, type="p")
chisq_gof_posthoc(X, type="chisq")

chisq_gof_posthoc(X, method="bonferroni")

X1 <- c(4, 3, 6, 19)
chisq_gof_posthoc(X1)
chisq_gof_posthoc(X1, typ="all")
chisq_gof_posthoc(X1, type="p")
chisq_gof_posthoc(X1, type="chisq")
chisq_gof_posthoc(X1, method="bonferroni")
```

```
chisq_gof_posthoc(X, method="bonferroni")

chisq_gof_posthoc(c(49, 30, 63, 59, 40, 60))

#' #'

## End(Not run)
```

---

`chisq_gof_posthoc_apa` *chisq_gof_posthoc_apa*

---

## Description

chisq_gof_posthoc_apa

## Usage

```
chisq_gof_posthoc_apa(df, adj = FALSE)
```

## Arguments

| | |
|---|---|
| `df` | df |
| `adj` | adj |

## Value

df

---

`chisq_multcomp2` *Pairwise comparisons after a chi-squared goodness-of-fit test*

---

## Description

Pairwise comparisons after a chi-squared goodness-of-fit test

## Usage

```
chisq_multcomp2(x, p.method = "fdr", ko = TRUE, sep = " / ")
```

## Arguments

| | |
|---|---|
| `x` | numeric vector (counts). |
| `p.method` | method for p-values correction. See help of p.adjust |
| `ko` | TRUE variable names is changed to korean language |
| `sep` | sep =" / " default |

### Value

result

### Functions

- `chisq_multcomp2()`: pairwise comparisons after a global chi-squared goodness-of-fit test.

### Examples

```
\#\# Not run:
#'
counts <- c(49,30,63,59)
chisq.test(counts)
chisq_multcomp2(counts)
# compare
chisq.multcomp2(counts)


## End(Not run)
```

---

## `chisq_pivot_table` *pivot*

---

### Description

pivot

### Usage

```
chisq_pivot_table(
  data,
  formula,
  names_from = NULL,
  unite = FALSE,
  col = 1:2,
  remove = TRUE,
  change_col = TRUE
)
```

### Arguments

| | |
|---|---|
| `data` | data |
| `formula` | formul |
| `names_from` | column |
| `unite` | 1,2 colum unite |
| `col` | 합쳐질컬럼 |
| `remove` | 기존것지우기 |
| `change_col` | 열의위치바꾸기 |

### Value

데이터

### Examples

```
\#\# Not run:

#' kge_bind3a %>%
filter(type == "고유어1")%>%
  pivot_table(~ area + age + 성조, unite=TRUE)
#'
kge_bind3a %>%
  filter(type == "고유어1")%>%
  pivot_table(~  age + area+ 성조, unite=FALSE)

#고유어1 공명음  지역별 연령
kge_bind3a %>%
  filter(type == "고유어1" & a1=="공명음")%>%
  pivot_table(~ age + area + 성조, unite=T)  #%>% md()



kge_bind3a %>%
  filter(type == "고유어1")%>%
  pivot_table(~  age +  성조, unite=FALSE)

#고유어1 공명음  지역별 연령
kge_bind3a %>%
  filter(type == "고유어1" & a1=="공명음")%>%
  pivot_table(~  area + 성조, unite=F)  #%>% md()



## End(Not run)
#'
```

---

## chisq_posthoc *chisq post hoc*

---

### Description

chisq post hoc

### Usage

```
chisq_posthoc(
  tbl,
  test = fisher.test,
  Rows = TRUE,
  control = c("bonferroni", "fdr", "BH", "BY", "holm", "hochberg", "hommel"),
  digits = 4,
  ...
)
```

### Arguments

| | |
|---|---|
| tbl | tavke |
| test | fusger.test, chisq.test |
| Rows | rows , FLASE = cols |
| control | p adjust"bonferroni","fdr","BH","BY","holm","hochberg","hommel" |
| digits | 4 |
| ... | ... |

### Value

result

### Examples

```
\#\# Not run:
as.table(cbind("T"=c(30,40,60,120),"S"= c(50,70,60,80)))%>%
  chisq_posthoc(Rows  = TRUE)

as.table(cbind("T"=c(30,40,60,120),"S"= c(50,70,60,80)))%>%
  chisq_posthoc(Rows  = FALSE)

as.table(cbind("T"=c(30,40,60,120),"S"= c(50,70,60,80)))%>%
  chisq_posthoc(test="chisq.test", Rows  = TRUE)

## End(Not run)
```

---

## chisq_retest *Functions to check the chi-square test of a paper*

---

### Description

Functions to check the chi-square test of a paper

### Usage

```
chisq_retest(
  input,
  ncol = 2,
  colname = NULL,
  rowname = NULL,
  type = "all",
  simulate.p.value = FALSE,
  digits = 3,
  correct = TRUE,
  B = 2000
)
```

### Arguments

| | |
|---|---|
| `input` | vectro |
| `ncol` | matrix col |
| `colname` | input |
| `rowname` | input |
| `type` | all, chisq |
| `simulate.p.value` | default FaLSE fisher exact test |
| `digits` | round 3 |
| `correct` | default is Pearson's Chi-squared test with Yates'continuity correction (If you have something with a small sample count, you should use it. ), FALSE Pearson's Chi-squared test |
| `B` | B is bootstrap default 2000 |

### Value

chisquare data

### Examples

```
\#\# Not run:
## Mina Choi. (2007). Analysis of learning effects
# according to online and offline learning environments
#  in the same class by the same instructor.
#  Educational Information Media Research, 13(4>  5-23.
## jjpaper
## https://blog.naver.com/shoutjoy/223005233887

chisq_retest(c(111, 33, 64, 58), ncol=2 )
## 논문에 나온 값이 잘못됨 :chisq = 17.793
## this chisq = 16.7

chisq_retest(c(111, 33, 64, 58), ncol=2, type="chisq" )

chisq_retest(c(111, 33, 64, 58), ncol=2,
  colname = c("experience", "Non-experience"),
  rowname =c("E-Leanring","FaceToFace"))

chisq_retest(c(4,10, 78, 46, 6, 0,5,79,36,2),
  ncol=2, simulate.p.value=T )

chisq_retest(c(4,10, 78, 46, 6, 0,5,90,36,2),
  ncol=2 ,simulate.p.value=T)

chisq_retest(c(4,10, 78, 46, 6, 0,5,79,36,2), ncol=2,
  colname = c("experience", "Non-experience"),
  rowname = c("maxlow","low","mid","high","maxhigh") )


##  Comparison of gender learning methods
# --> An invalid value was calculated.
## chisq = 5.238, p =0.015
```

```
# But recalculate chisq = 4.69, p = 0.0303
chisq_retest(c(84, 60, 54, 68), ncol=2 ,
             colname = c("experience", "Non-experience"),
                         rowname = c("male", "female"))

## Comparing Learner Characteristics by Grade Level, Mina Choi (2007)

chisq_retest(c(18,38, 40, 48, 52,35,19,16), ncol=2 ,
colname = c("experience", "Non-experience"),
rowname = c("1학년","2학년","3학년","4학년")


##If the value entered is not a vector value, but a value calculated with table(),
# it is automatically calculated with the ....
table(mtcars$am, mtcars$vs) %>% Chisq_retest()


## End(Not run)
```

---

`chisq_table_sum` *chisq table resize*

---

## Description

chisq table resize

## Usage

```
chisq_table_sum(data, ..., trans = FALSE, sep = "_")
```

## Arguments

| | |
|---|---|
| `data` | mat |
| `...` | con 1,1,2,2 |
| `trans` | matr transpose |
| `sep` | sep rowname combine ”-” |

## Value

mat

## Examples

```
\#\# Not run:
#'
# Test data
data <- data.frame(T = c(30, 40, 60, 120),
                   S = c(50, 70, 60, 80))
rownames(data) <- LETTERS[1:4]
data <- as.matrix(data)
```

```
# Example usage
print(chisq_table_sum(data, 1, 1, 2, 2))

chisq_table_sum(data, 1, 1, 2, 2)
chisq_table_sum(data, con = c(1, 1, 2, 2))
chisq_table_sum(data, la = c(1, 1, 2, 2))


## End(Not run)
```

---

`chisq_test` *chisq_test*

---

## Description

chisq_test

## Usage

```
chisq_test(
  x,
  y = NULL,
  correct = TRUE,
  p = rep(1/length(x), length(x)),
  rescale.p = FALSE,
  simulate.p.value = FALSE,
  B = 2000
)
```

## Arguments

| | |
|---|---|
| `x` | a numeric vector or matrix. x and y can also both be factors. |
| `y` | a numeric vector; ignored if x is a matrix. If x is a factor, y should be a factor of the same length. |
| `correct` | a logical indicating whether to apply continuity correction when computing the test statistic for 2 by 2 tables: one half is subtracted from all O−E differences ; however, the correction will not be bigger than the differences themselves. No correction is done if simulate.p.value = TRUE. |
| `p` | a vector of probabilities of the same length as x. An error is given if any entry of p is negative. |
| `rescale.p` | a logical scalar; if TRUE then p is rescaled (if necessary) to sum to 1. If rescale.p is FALSE, and p does not sum to 1, an error is given. |
| `simulate.p.value` | a logical indicating whether to compute p-values by Monte Carlo simulation |
| `B` | an integer specifying the number of replicates used in the Monte Carlo test. |

### Value

A list with class "htest" containing the following components:

### Examples

```
\#\# Not run:

## From Agresti(2007) p.39
M <- as.table(rbind(c(762, 327, 468), c(484, 239, 477)))
dimnames(M) <- list(gender = c("F", "M"),
party = c("Democrat","Independent", "Republican"))
(Xsq <- chisq.test(M))  # Prints test summary
Xsq$observed   # observed counts (same as M)
Xsq$expected   # expected counts under the nul
Xsq$residuals  # Pearson residuals
Xsq$stdres     # standardized residuals


x <- matrix(c(12, 5, 7, 7), ncol = 2)
chisq_test(x)$p.value           # 0.4233
chisq_test(x, simulate.p.value = TRUE, B = 10000)$p.value

## End(Not run)
```

---

## chisq_test_By1 *one sample chisq test by group*

---

### Description

one sample chisq test by group

### Usage

```
chisq_test_By1(dataset, sel = NULL, correct = FALSE, simulate.p.value = FALSE)
```

### Arguments

| | |
|---|---|
| dataset | data.frame |
| sel | select column group |
| correct | chisq correct |
| simulate.p.value | simulate.p.value |

### Value

Chi-square results by group

### Examples

```
\#\# Not run:
mtcars%>%chisq_test_By1("cyl")

mtcars%>%chisq_test_By1("am")
iris%>%chisq_test_By1("Species")

## End(Not run)
```

---

`chisq_test_By2` *twosample chisq.test by group*

---

### Description

twosample chisq.test by group

### Usage

```
chisq_test_By2(
  data,
  v1,
  v2,
  sel,
  correct = TRUE,
  simulate.p.value = FALSE,
  warning = FALSE,
  type = "all"
)
```

### Arguments

| | |
|---|---|
| `data` | data,frame |
| `v1` | first variable |
| `v2` | second variable |
| `sel` | group |
| `correct` | correct |
| `simulate.p.value` | simulate.p.value |
| `warning` | warning |
| `type` | ”all”, ”res” |

### Value

chisq result

### Examples

```
\#\# Not run:

mtcars%>%  chisq_test_By2("am", "vs", "cyl")

mtcars%>% chisq_test_By2("am", "vs", sel= "cyl")

Mtcars <- mtcars  # 데이터를 원본과 겹치지 않도록 복사
Mtcars$am = factor(Mtcars$am, levels=c(0,1), labels= c("automatic","manual" ))
Mtcars$vs = factor(Mtcars$vs, levels=c(0,1), labels= c("V-shaped","straight" ))
Mtcars$cyl = factor(Mtcars$cyl, levels=c(4,6,8),
             labels= c("cyl-4","cyl-6","cyl-8"))

chisq_test_By2(data = Mtcars, v1 = "am", v2 = "vs", sel = "cyl")


## End(Not run)
```

---

## chisq_test_By3 그룹별카이제곱분석

---

### Description

그룹별카이제곱분석

### Usage

```
chisq_test_By3(data, grp, formula, raw = FALSE)
```

### Arguments

| | |
|---|---|
| data | data |
| grp | group variable |
| formula | ~ vq +v2 |
| raw | warning |

### Value

table

### Examples

```
\#\# Not run:
#'
chisq_test_By3(mtcars, cyl, ~ am+vs )
chisq_test_By3(mtcars, cyl, ~ am+vs , raw=T)


## End(Not run)
```

chisq_test_cell *Chi-square analysis by cell*

## Description

Chi-square analysis by cell

## Usage

```
chisq_test_cell(data, type = "res", trans = FALSE, plot = TRUE, digits = 3)
```

## Arguments

| | |
|---|---|
| data | table |
| type | res, alll()default), chisq_result , observed expected, residuals , chisq_cell , stdres, chisq_cell_sig , cramersv , OE_ratio, OE_sig , graph |
| trans | plot transpose |
| plot | TRUE |
| digits | digits= 3 |

## Value

chisq

## Examples

```
\#\# Not run:
#'


M <- as.table(rbind(c(762, 327, 468), c(484, 239, 477)))
dimnames(M) <- list(gender = c("F", "M"),
                    party = c("Democrat","Independent", "Republican"))
M %>% chisq_test_cell()

#notsig
examtest  = matrix(c(7,14,13,6), nrow=2,
                   dimnames= list(c("사전검사","사후검사"),
                                  c("합격","불합격")))
examtest
fisher.test(examtest)

examtest %>% chisq_test_cell()

## End(Not run)
```

chisq_test_cor *kge_chisq_table chisq table observed/Expected table-*

## Description

kge_chisq_table chisq table observed/Expected table-

## Usage

```
chisq_test_cor(
  dataset,
  v1 = "var1",
  v2 = "var2",
  title = "Table",
  type = "result",
  digits = 3,
  yadd = 0.4,
  Ncol = NULL,
  trans = FALSE,
  simple = FALSE,
  ko = TRUE,
  simulate.p.value = FALSE,
  correct = FALSE,
  size_bartext = 5,
  strip_size = 16,
  axis_size = 16,
  text_size = 13,
  xlab = "관측기대비율",
  ylab = "var2",
  cramer = "adjust"
)
```

## Arguments

| | |
|---|---|
| dataset | data.frame |
| v1 | col |
| v2 | row |
| title | title |
| type | res2 default |
| digits | 3 |
| yadd | up yadd |
| Ncol | grid |
| trans | transpose |
| simple | * style |
| ko | graph korean |
| simulate.p.value | exact |
| correct | exactt |

| | |
|---|---|
| size_bartext | graph text |
| strip_size | strip |
| axis_size | axis_size |
| text_size | text_size |
| xlab | xlab |
| ylab | ylab |
| cramer | cramer cor |

## Value

multiple data

## Examples

```
\#\# Not run:
kge_chisq_table(mtcars,"am","cyl")


#chisq test result
vote_chi <- Chisq_retest(c(183, 35, 824, 50), ncol=2,
                         colname = c("Did not vote", "Voted"),
                          rowname =c("All other ages","Youngest"))

vote_chi


# generate matrix
voteMat = matrix(c(183, 35, 824, 50), ncol=2)
colnames(voteMat) = c("Did not vote", "Voted")
rownames(voteMat) = c("All other ages","Youngest")

voteMat%>% data.frame() %>%
  long_df("voted", rowname="youth")%>% #long format
  add_rows_freq() %>%     # add rows
  kge_chisq_table("voted","youth")    #chisq test


#신진영(2013)
shin2013_mat =matrix(c(14,1,23,3,4,0,9,0,13,18,89,8,11,40,46,12,3,4,22,0), ncol= 5)
# colnames(shin2013_mat)=c("지능지수가높다","학업성취도가높다","창의적사고를한다","특정분야에서뛰어난
colnames(shin2013_mat)=c("지능지수","학업성취도","창의적사고","특정분야","모든분야")
rownames(shin2013_mat)=c("고졸","초대졸","대졸","대학원이상")
shin2013_mat

shin2013_mat %>% data.frame() %>%
  rownames_to_column("학력") %>%
  to_long(names_to = "영재성",cols=2:6) %>%
  unCount() %>%
  chisq_test_cor(v1="영재성", v2="학력")


shin2013_mat%>%  data.frame() %>%
  long_df("학력","영재성") %>%
  unCount() %>%
```

```
  chisq_test_cor("학력","영재성")

## End(Not run)
```

---

`chisq_test_kge` *kge_chisq_table chisq table observed/Expected table-*

---

## Description

kge_chisq_table chisq table observed/Expected table-

## Usage

```
chisq_test_kge(
  dataset,
  v1 = "var1",
  v2 = "var2",
  title = "Table",
  type = "result",
  digits = 3,
  yadd = 0.4,
  Ncol = NULL,
  trans = FALSE,
  simple = FALSE,
  ko = TRUE,
  simulate.p.value = FALSE,
  correct = FALSE,
  size_bartext = 5,
  strip_size = 16,
  axis_size = 16,
  text_size = 13,
  xlab = "관측기대비율",
  ylab = "var2",
  cramer = "adjust"
)
```

## Arguments

| | |
|---|---|
| `dataset` | data.frame |
| `v1` | col |
| `v2` | row |
| `title` | title |
| `type` | res2 default |
| `digits` | 3 |
| `yadd` | up yadd |
| `Ncol` | grid |
| `trans` | transpose |
| `simple` | * style |

| | |
|---|---|
| ko | graph korean |
| simulate.p.value | exact |
| correct | exactt |
| size_bartext | graph text |
| strip_size | strip |
| axis_size | axis_size |
| text_size | text_size |
| xlab | xlab |
| ylab | ylab |
| cramer | cramer cor |

## Value

multiple data

## Examples

```
\#\# Not run:
kge_chisq_table(mtcars,"am","cyl")


#chisq test result
vote_chi <- Chisq_retest(c(183, 35, 824, 50), ncol=2,
                         colname = c("Did not vote", "Voted"),
                          rowname =c("All other ages","Youngest"))

vote_chi


# generate matrix
voteMat = matrix(c(183, 35, 824, 50), ncol=2)
colnames(voteMat) = c("Did not vote", "Voted")
rownames(voteMat) = c("All other ages","Youngest")

voteMat%>% data.frame() %>%
  long_df("voted", rowname="youth")%>% #long format
  add_rows_freq() %>%     # add rows
  kge_chisq_table("voted","youth")    #chisq test


#신진영(2013)
shin2013_mat =matrix(c(14,1,23,3,4,0,9,0,13,18,89,8,11,40,46,12,3,4,22,0), ncol= 5)
# colnames(shin2013_mat)=c("지능지수가높다","학업성취도가높다","창의적사고를한다","특정분야에서뛰어난
colnames(shin2013_mat)=c("지능지수","학업성취도","창의적사고","특정분야","모든분야")
rownames(shin2013_mat)=c("고졸","초대졸","대졸","대학원이상")
shin2013_mat

shin2013_mat %>% data.frame() %>%
  rownames_to_column("학력") %>%
  to_long(names_to = "영재성",cols=2:6) %>%
  unCount() %>%
```

```
  chisq_test_kge(v1="영재성", v2="학력")


shin2013_mat%>%  data.frame() %>%
  long_df("학력","영재성") %>%
  unCount() %>%
  chisq_test_kge("학력","영재성")


## End(Not run)
```

---

`cmh_test` *cochran_mantel_haenszel_test*

---

## Description

cochran_mantel_haenszel_test

## Usage

```
cmh_test(
  x,
  y = NULL,
  z = NULL,
  alternative = c("two.sided", "less", "greater"),
  correct = TRUE,
  exact = FALSE,
  conf.level = 0.95
)
```

## Arguments

| | |
|---|---|
| `x` | array |
| `y` | NULL |
| `z` | NULL |
| `correct` | TRUE |
| `exact` | FALSE |
| `conf.level` | 0.95 |
| `alternativec("two.sided",` | ”less”, ”greater”) |

## Value

result

### Examples

```
\#\# Not run:

CAD <-array(c(8, 4, 10, 11, 21, 6, 9, 9), dim = c(2, 2, 2),
            dimnames = list(
              EKG = c(">=0.1 ST Dep", "< 0.1 ST Dep"),
              Response = c("Case", "Control"),
              Penicillin.Level = c("Female", "Male")))
CAD

#Example: Coronary Artery Disease
mantelhaen.test(CAD,correct=FALSE)

cmh_test(CAD,correct=FALSE)%>%tidy()


## End(Not run)
```

---

`col2row` *column to rownames col2row*

---

### Description

column to rownames col2row

### Usage

```
col2row(data, col = 1)
```

### Arguments

| | |
|---|---|
| `data` | data.frame |
| `col` | Columns that should be rows |

### Value

data

### Examples

```
\#\# Not run:
#'
data_text = "Tobacco_Dependence (0,5] (5,10] (10,15] (15,20] (20,98]
No_Nicotine_Dependence 130 210 43 114 20
Nicotine_Dependence 119 267 91 254 67"
#method1
# nt = data_text%>% text2df(header=T)%>%
#         tibble::column_to_rownames("Tobacco_Dependence")
#method2
nt = data_text%>% text2df(header=T)%>% jjstat::col2row()
nt=Char2num(nt,1:5)
nt
```

```
nt %>%tibble()

#NEW data
New = data.frame(
  rownames= c('No_Nicotine_Dependence', 'Nicotine_Dependence'),

  "A" = c(130, 119),
  "B" = c(210, 267),
  "C" = c(43, 91),
  "D" = c(114, 254),
  "E" = c(20, 67)
);New
New%>%col2row()
New%>%col2row(col="B")
#mtcars
data_mtcars=mtcars%>%tibble::rownames_to_column()
data_mtcars%>%col2row()




## End(Not run)
```

---

`collapse_path` *Extract only the corresponding paths from the data with the given material and the given paths*

---

## Description

Extract only the corresponding paths from the data with the given material and the given paths

## Usage

```
collapse_path(data, path_string, est = "Original")
```

## Arguments

| | |
|---|---|
| `data` | boot |
| `path_string` | path IMAG -> EXPE -> VAL |
| `est` | Original, Mean.Boot |

## Value

paths

**Examples**

```
\#\# Not run:
#'

# Example usage
sdata <- data.frame(
  paths = c("IMAG -> EXPE", "IMAG -> SAT", "IMAG -> LOY",
   "EXPE -> QUAL", "EXPE -> VAL", "EXPE -> SAT",
            "QUAL -> VAL", "QUAL -> SAT",
            "VAL -> SAT", "SAT -> LOY"),
  Original = c(0.578959128, 0.200724200, 0.275149576,
   0.848344408, 0.105477650, -0.002753995,
               0.676655529, 0.122144566, 0.589330650, 0.495479322),
  Mean.Boot = c(0.58232765, 0.21209094, 0.27486564,
  0.84793899, 0.10992051, -0.01008222,
                0.67780527, 0.15132543, 0.56312717, 0.50757755),
  Std.Error = c(0.04358293, 0.06056988, 0.07729000,
  0.01864573, 0.06968826, 0.06507903,
                0.07760527, 0.08440236, 0.08723293, 0.07856694),
  perc.025 = c(0.497644719, 0.107561558, 0.140641502,
   0.815794494, -0.025566471, -0.147788188,
               0.519959683, -0.008629516, 0.392664443, 0.351774130),
  perc.975 = c(0.6646198, 0.3275312, 0.4161131,
   0.8828875, 0.2545785, 0.1086856,
               0.8166486, 0.2991045, 0.7012494, 0.6556333)
)
collapse_path(sdata, path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY")
## paths  Original  Std.Error
## 1 IMAG -> EXPE 0.5789591 0.04358293
## 2  EXPE -> VAL 0.1054776 0.06968826
## 3   VAL -> SAT 0.5893307 0.08723293
## 4   SAT -> LOY 0.4954793 0.07856694


#활용방법
collapse_path(sdata,
path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY")%>%
find_paths()%>%
find_paths_cal()

#'
## End(Not run)
```

---

`collapse_path2` 소벡테스트에입력될값으로산정하여 *list* 로출력

---

**Description**

소벡테스트에입력될값으로산정하여 list 로출력

### Usage

```
collapse_path2(data, path_string, show = FALSE, est = "Original")
```

### Arguments

| | |
|---|---|
| data | boot |
| path_string | paths |
| show | message |
| est | Original, Mean.Boot |

### Value

data

### Examples

```
\#\# Not run:
#'

# Example usage
sdata <- data.frame(
  paths = c("IMAG -> EXPE", "IMAG -> SAT", "IMAG -> LOY",
   "EXPE -> QUAL", "EXPE -> VAL", "EXPE -> SAT",
            "QUAL -> VAL", "QUAL -> SAT",
            "VAL -> SAT", "SAT -> LOY"),
  Original = c(0.578959128, 0.200724200, 0.275149576,
   0.848344408, 0.105477650, -0.002753995,
              0.676655529, 0.122144566, 0.589330650, 0.495479322),
  Mean.Boot = c(0.58232765, 0.21209094, 0.27486564,
  0.84793899, 0.10992051, -0.01008222,
               0.67780527, 0.15132543, 0.56312717, 0.50757755),
  Std.Error = c(0.04358293, 0.06056988, 0.07729000,
  0.01864573, 0.06968826, 0.06507903,
               0.07760527, 0.08440236, 0.08723293, 0.07856694),
  perc.025 = c(0.497644719, 0.107561558, 0.140641502,
   0.815794494, -0.025566471, -0.147788188,
              0.519959683, -0.008629516, 0.392664443, 0.351774130),
  perc.975 = c(0.6646198, 0.3275312, 0.4161131,
   0.8828875, 0.2545785, 0.1086856,
              0.8166486, 0.2991045, 0.7012494, 0.6556333)
)
#check 1
collapse_path2(sdata, path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY")

#check 2
collapse_path2(sdata, path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY", show=TRUE)
#'
# TEST

collapse_path2(sdata, path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY")
collapse_path2(sdata, path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY", show=TRUE)
collapse_path2(satpls_boot, path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY", show=TRUE)

# 소벨테스트
```

```
sobel_test_extend(coefficients= c("IMAG -> EXPE" = 0.578959128,
                                  "EXPE -> SAT" = -0.002753995,
                                  "SAT -> LOY" = 0.495479322),
                  se_values = c(se1 = 0.04358293,
                                se2 = 0.06507903,
                                se3 = 0.0785669))


Resdata  =   collapse_path2(sdata, path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY")

sobel_test_extend(coefficients= Resdata$coefficients, Resdata$se_values)


sobel_test_extend(
  coefficients= collapse_path2(sdata,
              path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY")$coefficients,
  se_values =  collapse_path2(sdata,
        path_string= "IMAG -> EXPE -> VAL -> SAT -> LOY")$se_values)


sobel_test_extend(coefficients= collapse_path2(sdata,
         path_string=  "IMAG -> EXPE -> SAT -> LOY")$coefficients,
         se_values =  collapse_path2(sdata, path_string= "IMAG -> EXPE -> SAT -> LOY")$
                   se_values, show=FALSE)


## End(Not run)
```

---

`colnames_by_dict` *colnames_by_dict: Rename only column names using a dictionary*

---

## Description

This function renames the column names of a data frame using a dictionary vector.

## Usage

```
colnames_by_dict(df, dict)
```

## Arguments

| | |
|---|---|
| `df` | A data.frame or tibble whose column names you want to rename. |
| `dict` | A character vector of key-value pairs. It can be either: - Named vector: c("old1" = "new1", "old2" = "new2") - Unnamed pairwise vector: c("old1", "new1", "old2", "new2") |

## Value

A data.frame with renamed column names.

### Examples

```
var_named <- c(
  "access","연구접근성",
  "barrier","진로장벽",
  "efficacy","연구효능감",
  "interest","진로흥미",
  "outcome","결과기대"
)

df <- data.frame(access = 1:2, barrier = 3:4, outcome = 5:6)
colnames_by_dict(df, var_named)
```

---

`colnames_change` *colnames_change: Change column names partially or entirely, with optional preservation of original names*

---

### Description

colnames_change: Change column names partially or entirely, with optional preservation of original names

### Usage

```
colnames_change(df, ...)
```

### Arguments

| | |
|---|---|
| `df` | A data frame. The target data frame whose column names need to be changed. |
| `...` | Character vector of new column names. Use an empty string (‘""‘) to keep the original column name for specific positions. |

### Value

A data frame with updated column names. Columns corresponding to ‘""‘ in the input retain their original names.

### Examples

```
\#\# Not run:
# Example with partial column renaming using mtcars
data(mtcars)
head(mtcars)

# Rename selected columns
mtcars_updated <- mtcars %>%
  colnames_change("Miles_per_Gallon", "Cylinders", "Displacement", "", "Weight")

head(mtcars_updated)

## End(Not run)
```

`colnames_change_df` *colnames_change_df: Change column names of a data frame within a specified range*

## Description

colnames_change_df: Change column names of a data frame within a specified range

## Usage

```
colnames_change_df(df, range = 1:ncol(df), impute = NULL)
```

## Arguments

| | |
|---|---|
| `df` | A data frame whose column names need to be changed. |
| `range` | A vector or range indicating the columns to change. It can be numeric (e.g., 1:27) or character specifying column names (e.g., "disp:vs"). |
| `impute` | A character vector of new column names to assign to the specified range. |

## Value

The data frame with updated column names.

## Examples

```
# Example 1: Change the first 11 columns of mtcars
new_colnames <- c(
  paste0("feature_", 1:6),
  paste0("metric_", 1:5)
)
updated_mtcars <- colnames_change_df(
  df = mtcars,
  range = 1:11,
  impute = new_colnames
)

# Example 2: Change a subset of columns in mtcars by name range
new_colnames <- c(
  paste0("feature_", 1:6)   # Replace columns "disp" to "vs"
)
updated_mtcars <- colnames_change_df(
  df = mtcars,
  range = "disp:vs",
  impute = new_colnames
)
```

---

`colnames_to_row` *colnames_to_row: Insert column names as a row in a data frame*

---

### Description

colnames_to_row: Insert column names as a row in a data frame

### Usage

```
colnames_to_row(df, row = 1)
```

### Arguments

| | |
|---|---|
| `df` | A data frame whose column names will be inserted as a row. |
| `row` | An integer specifying the row number where column names should be inserted. Default is 1. |

### Value

A data frame with column names inserted as a row at the specified position.

### Examples

```
\#\# Not run:
df <- data.frame(a = 1:3, b = 4:6, c = 7:9)
colnames_to_row(df)
colnames_to_row(df, row = 2)
mtcars
mtcars %>% nrow()
mtcars %>% colnames_to_row(1)
mtcars %>% colnames_to_row(3)
mtcars %>% colnames_to_row(32)

## End(Not run)
```

---

`col_capital` *col_capital: Convert letters in a selected column to upper or lower case*

---

### Description

col_capital: Convert letters in a selected column to upper or lower case

### Usage

```
col_capital(df, col, direction = TRUE)
```

### Arguments

| | |
|---|---|
| `df` | A data.frame. The input dataset. |
| `col` | A column name (character) or index (numeric) indicating which column to transform. |
| `direction` | Logical. If TRUE (default), convert to upper case. If FALSE, convert to lower case. |

### Value

A data.frame with the selected column transformed to upper or lower case.

### Examples

```
\#\# Not run:
df <- data.frame(
  구분 = c("chisq", "df", "pvalue"),
  criterian = c("Chisq", "df", "p > .05"),
  Value = c("1732.243", "269", "0")
)

# Use column name
col_capital(df, "criterian", TRUE)

# Use column index
col_capital(df, 2, FALSE)

## End(Not run)
```

---

## `col_to_colnames` *one select column to colnames*

---

### Description

one select column to colnames

### Usage

```
col_to_colnames(df, col = 1, remove = TRUE)
```

### Arguments

| | |
|---|---|
| `df` | data frame |
| `col` | column to use as new column names (default = 1) |
| `remove` | logical. If TRUE, removes the selected column (default = TRUE). |

### Value

data frame with updated column names

### Examples

```
\#\# Not run:
# Example usage
df <- data.frame(
  요인 = c("에듀테크", "학습몰입", "지각된편리성", "지각된유용성", "태도", "지속사용"),
  SUX = c(NA, "0.948*", "0.933*", "0.947*", "0.902*", "0.865*"),
  FLOW = c(NA, NA, "0.951*", "0.967*", "0.929*", "0.876*"),
  PEOU = c(NA, NA, NA, "0.992*", "0.972*", "0.940*"),
  PU = c(NA, NA, NA, NA, "0.956*", "0.927*"),
  ATT = c(NA, NA, NA, NA, NA, "0.972*")
)

# col_to_colnames 실행
new_df <- col_to_colnames(df, col = 1, remove = FALSE)
print(new_df)

new_df2 <- col_to_colnames(df, col = 1, remove = TRUE)
print(new_df2)

## End(Not run)
```

---

## col_to_rownames *column to rownames , col_to_rownames*

---

### Description

column to rownames , col_to_rownames

### Usage

```
col_to_rownames(data, col = 1)
```

### Arguments

| | |
|---|---|
| data | data.frame |
| col | Columns that should be rows |

### Value

data

### Examples

```
\#\# Not run:
#'
data_text = "Tobacco_Dependence (0,5] (5,10] (10,15] (15,20] (20,98]
No_Nicotine_Dependence 130 210 43 114 20
Nicotine_Dependence 119 267 91 254 67"
#method1
# nt = data_text%>% text2df(header=T)%>%
#         tibble::column_to_rownames("Tobacco_Dependence")
#method2
```

```
nt = data_text%>% text2df(header=T)%>% jjstat::col2row()
nt=Char2num(nt,1:5)
nt
nt %>%tibble()

#NEW data
New = data.frame(
  rownames= c('No_Nicotine_Dependence', 'Nicotine_Dependence'),

  "A" = c(130, 119),
  "B" = c(210, 267),
  "C" = c(43, 91),
  "D" = c(114, 254),
  "E" = c(20, 67)
);New
New%>%col_to_rownames()

#mtcars
data_mtcars=mtcars%>%tibble::rownames_to_column()
data_mtcars%>%col_to_rownames()



## End(Not run)
```

---

`combine_data` *A function that combines two matrices of data, for example, to combine observation/expectation tables and p-values.*

---

## Description

A function that combines two matrices of data, for example, to combine observation/expectation tables and p-values.

## Usage

```
combine_data(mat_observed_expected, mat_p_value, left = " ", right = "")
```

## Arguments

| | |
|---|---|
| `mat_observed_expected` | matrix observed/expected |
| `mat_p_value` | matirx pvalue |
| `left` | ”(” etc input |
| `right` | ”)” |